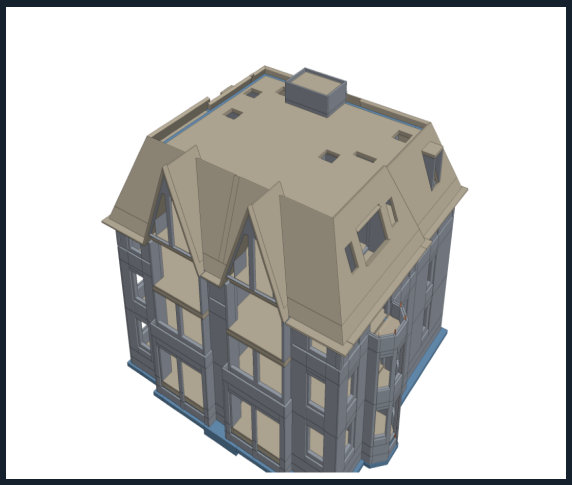
CV5570 | PROJECT REPORT

BIM-Based Construction Progress Monitoring via Drone Simulation

What we built, why we built it, and how every part works. Written so that any team member can explain the whole project afterwards, including the background concepts from scratch.



November 2026

Contents

1. The project in a few pages **5**

1.1 What it is . . . 5
1.2 The idea in six steps . . . 5
1.3 What we found . . . 6
1.4 The project in 30 seconds and in 2 minutes . . . 6

2. Background concepts, from scratch **7**

2.1 Construction progress monitoring . . . 7
2.2 BIM and IFC . . . 7
2.3 Point clouds and how they are captured . . . 8
2.4 Coordinate frames and rigid transforms . . . 8
2.5 Registration and ICP . . . 9
2.6 Nearest-neighbour search and KD-trees . . . 10
2.7 Finding the rough position: correlation and the FFT . . . 11
2.8 Ray casting and why we simulate . . . 11
2.9 Photogrammetry, SfM and COLMAP . . . 12
2.10 Segmentation, SAM and DART . . . 13
2.11 How we measure success . . . 13

3. The problem, the data and the system **14**

3.1 Goal and scope . . . 14
3.2 The test building . . . 14
3.3 System overview . . . 15

4. The pipeline, step by step **16**

4.1 P1: from IFC to a working table . . . 16
4.2 P2: the 16 construction stages and the answer key . . . 16
4.3 P3: simulating the scans . . . 17
4.4 Registration: laying the scan on the design . . . 20
4.5 Matching scan points to design elements . . . 22
4.6 The visibility check: not built, or just not seen? . . . 23
4.7 P7: status per element and the progress number . . . 24
4.8 R5: error bars on the progress number . . . 24

5. Experiments: colour, AI clutter removal and COLMAP **26**

5.1 Colour . . . 26
5.2 AI clutter removal with DART . . . 27
5.3 COLMAP: point clouds from drone photos . . . 28

6. Results and how we got here **32**

6.1 What each change did . . . 32

6.2 Timeline of the work . . . 34
6.3 Things we tried that did not work . . . 34
6.4 Found by looking carefully . . . 34

7. Limitations and honest caveats **35**

8. What is left, and what we would do next **35**

9. Using the repository **37**

9.1 Where things are . . . 37
9.2 How to run it . . . 37
9.3 Conventions . . . 38

10. Likely questions, with honest answers **38**

Appendix A. Glossary **40**

Appendix B. Per-stage results on the hardest test **42**

Appendix C. Script reference **43**

Appendix D. Project history from the git log **44**

About this report

This report explains one project from beginning to end: a system that decides how much of a building has been built by comparing a 3D scan of the construction site with the building's design model, and reports the project's progress in percent. It is written for the people on the team. Each of us worked on some parts (the simulator, the alignment, the experiments, the scoring). The goal is that anyone who reads this can explain **what** the project is, **why** each design choice was made, and **how** it works, including the parts they did not build themselves.

How to read it

- **In a hurry (20 minutes):** read Chapter 1 (the project on a few pages) and Chapter 6 (results), then skim the glossary.
- **New to the topic:** read Chapter 2 first. It explains BIM, point clouds, ICP, KD-trees, photogrammetry and COLMAP from scratch, with small pictures you can follow by eye.
- **To understand one part of the code:** go to Chapter 4 (one section per pipeline step) and Chapter 9 (where every file lives and how to run it).
- **To answer questions at a review:** Chapter 10 has likely questions with short honest answers.

Conventions used

- **Element** = one wall, slab, beam or column of the building. **Stage** = one of the 16 construction states we simulate.
- **Points** (as in "2 points of error") means *percentage points*. If the true progress is 60% and we estimate 58%, the error is 2 points.
- File names and commands are written in `monospace` and are relative to the repository root.

- Every number in this report comes from a result table in the repository (named where it is used). Nothing is rounded up in our favour; where something did not work, we say so.

1. The project in a few pages

1.1 What it is

Construction managers need to know, week by week, how far a building has got: which walls, floors and columns are already standing. Today this is done by people walking around with checklists and photos. It is slow and does not scale to big or fast-moving sites. A **drone** or a **laser scanner** can capture the whole site in minutes as a **point cloud** (millions of 3D dots on the visible surfaces). Meanwhile the building's **design model** (a BIM) says exactly where every element should be. If we could lay the scan on top of the design and check, element by element, whether there are scan points where the element should be, we would know what is built, and the overall progress follows by adding up.

That is this project: **automatic, element-level progress monitoring by matching a scan to its design model**. We build and test it entirely in simulation, because no public data set gives us a real scan together with the exact truth about what was built at that moment.

1.2 The idea in six steps

1. **Take a real design.** We use the design model of a Dutch apartment block (Schependomlaan): 1,355 elements with geometry.
2. **Un-build it into 16 stages** (foundation, then for each floor: structure, outer walls, inner walls). For every stage we know exactly which elements exist. This is our *answer key*.
3. **Simulate a scan of each stage.** A virtual drone flies around the building and a virtual scanner walks through each floor, producing a point cloud. We add realistic problems: sensor noise, holes, a scaffold-and-crane site, workers, elements built a little wrong, and a scan that starts metres away from the design.
4. **Align the scan with the design** (registration). This is harder than it sounds, and our new method is the biggest technical improvement since the mid-term.
5. **Match scan points to elements** and decide for each element: completed, partly built, not started, or *uncertain* (not enough view to tell).
6. **Add up** to a progress percentage with a range, and compare with the answer key.

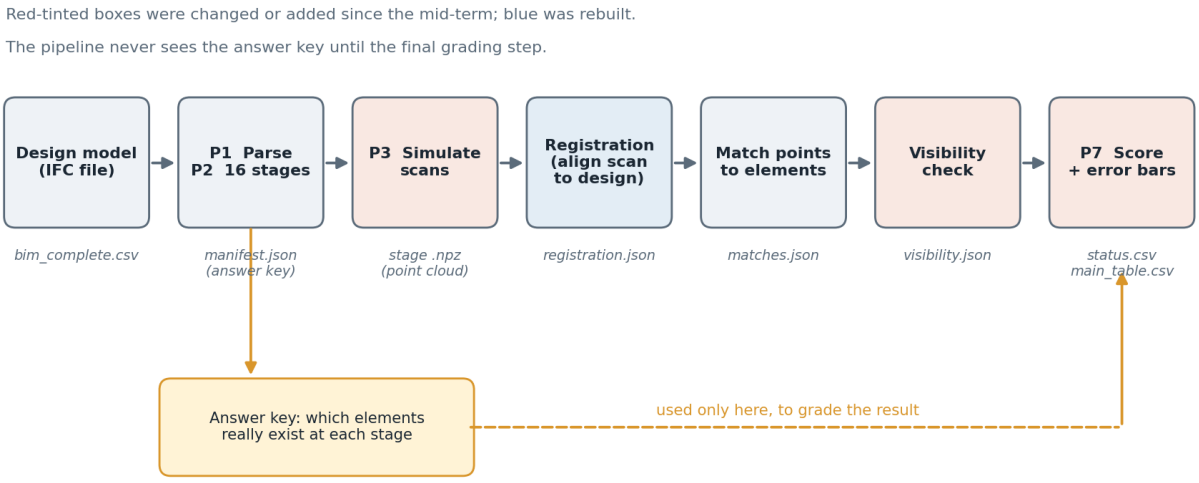


Figure 1. *The whole pipeline. Files in italics are what one step hands to the next. The answer key (amber) is used only for the final grading.*

1.3 What we found

Question	Answer	Source
How accurate is the progress number on an easy, clean test?	Average error 0.8 points (it was 4.8 for the first pipeline)	improvement_ladder.csv
On the hardest test (realistic sensor, busy site, late and misplaced elements)?	Average error 2.0 points; worst stage 5.7	main_table_13
With the learned correction and a 90% range?	Average error 0.7 points; the truth is inside the range at 15 of 16 stages	main_table_13
How well did the old ICP alignment do on the hard test?	Average error 40 points, and it never raised a warning	main_table_busyold
Can the cloud come from drone photos (COLMAP) instead?	Mid-construction: error 5 points. Finished building: error 35 points	main_table_sfm
Does colour help alignment?	Not measurably. All test cases were already right using geometry alone	r3_registration_summary_color.md
Does AI clutter removal (DART) help?	It removes 41% of clutter points and keeps 99.6% of structure, but the progress estimate did not change (one stage only)	PROGRESS.md

Table 1. *Headline results. All errors are in percentage points, averaged over the 16 stages.*

The honest limits

Everything here is simulation on **one building**; no real survey has been run yet. About a quarter of all elements are never seen by our camera plans even when built, so they are reported as *uncertain*. Clutter such as scaffold decks and workers still fools the matching in some places (28 of 67 false "built" calls on the busy site). Thresholds were tuned on these runs; a frozen test split is still to be made.

1.4 The project in 30 seconds and in 2 minutes

30 seconds

We check construction progress by laying a 3D scan of the site on the building's design model and testing, element by element, whether the scan has points where the element should be. We test it on a real building design that we 'un-build' into 16 stages, so we know the true progress exactly. Our pipeline reaches about 1 to 2 points of error even with realistic noise, a busy site and a scan that starts metres off.

2 minutes

The design is a BIM: every wall, slab, beam and column with a position and a unique ID. We cut it into 16 stages and simulate a drone and indoor scanner at each. The scan starts in the wrong place (like GPS), so we first align it: a floor-plan search finds the rough position, ICP fine-tunes it. Then every scan point is given to its nearest design element, and elements with enough points count as built. If an element has no points we ask a second question: would the cameras even have seen it? If not, it is *uncertain*, not *not built*.

The elements add up to a progress percentage. A learned correction removes the typical bias and gives a 90% range. We tested colour, an AI clutter-removal model and COLMAP photogrammetry; none replaced the main method, but each taught us something.

2. Background concepts, from scratch

This chapter explains every idea the project relies on, assuming no background. Each section ends with a short box in plain words. Skip what you already know.

2.1 Construction progress monitoring

A building is built in a sequence: foundation, structure (columns, beams, floor slabs), outer walls, inner walls, finishing. Owners and contractors compare the real state with the schedule to catch delays early. Doing that by eye is slow and subjective. Automation needs two things: a way to capture the current state (scans or photos) and a reference of what the finished building should be (the design). The core difficulty is that **the capture is messy**: noisy, incomplete, full of things that are not in the design (scaffolding, cranes, workers, piles of material), and positioned in its own coordinate system.

2.2 BIM and IFC

BIM (Building Information Modelling) means a digital model in which every part of the building is an object with geometry and information: a wall knows its shape, its material, the storey it belongs to. **IFC** (Industry Foundation Classes) is the open file format for exchanging BIMs. An IFC file is text in which each line defines an object, for example an `IfcWall`, `IfcSlab`, `IfcBeam` or `IfcColumn`, plus relations such as "this wall is contained in storey 2".

- Each object has a **GlobalId**: a 22-character unique identifier. We use it as the name of an element everywhere.
- The geometry of each element is a **triangle mesh**: a surface made of many small triangles. We can compute its bounding box, volume and centre.
- Our building: 1,692 structural elements are declared in the IFC (934 walls plus 282 standard-case walls, 23 columns, 174 beams, 279 slabs), and **1,355 of them have usable geometry**: 879 walls, 279 slabs, 174 beams, 23 columns, about 1,012 cubic metres in total. Those 1,355 are the working set for everything in this report.

> **Quirks we had to handle**
> In IFC2X3, `IfcWallStandardCase` is a sub-type of `IfcWall`, so asking for walls already returns both. The storey of an element must be read from the "contained in" relation, not from the storey's `Elevation` field (that field is in millimetres while the geometry is in metres). The data set also has weekly *as-planned* IFC files, but their GlobalIds do not match the design file at all (0% overlap), so only the design IFC can be used as truth.

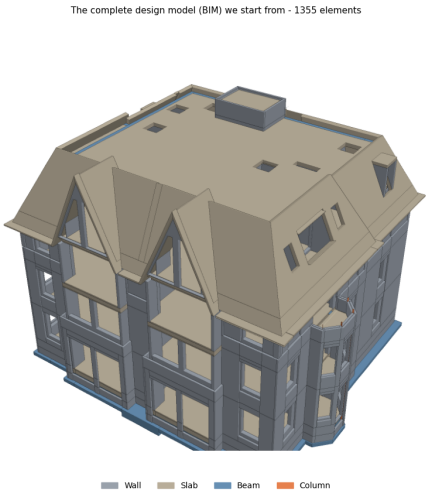


Figure 2. *The complete design model of the test building, coloured by element type.*

2.3 Point clouds and how they are captured

A **point cloud** is a list of 3D points (x, y, z), often with a colour, that lie on the surfaces a sensor could see. It has no walls, no objects, no meaning: just dots. Real clouds come from three families of sensors:

- **Laser scanners (LiDAR).** A device measures the distance to a surface by timing a laser pulse. Terrestrial scanners on a tripod give dense, accurate points but only where they can see from the places they stand. Indoor handheld versions work like a scanner you walk with.
- **Photogrammetry.** A drone takes many overlapping photos; software works out the 3D structure from the pictures (Section 2.9). This is cheap and covers the outside of a building well, but struggles with plain surfaces, glass and thin objects.
- **SLAM.** A moving sensor builds a map and tracks its own position at the same time. Its map lives in its own start frame.

A cloud is never perfect. Typical problems are **noise** (points scatter around the true surface), **holes** (surfaces the sensor could not see or measure), **occlusion** (a thing in front hides a thing behind), **clutter** (things that are not in the design) and **an unknown position** (Section 2.4).

2.4 Coordinate frames and rigid transforms

Every cloud and every model lives in a **coordinate frame**: an origin, three axes and a unit. The design model uses one frame. A drone survey reports points in a frame fixed by its GPS and compass; an indoor scanner reports them in the frame where it was switched on. To compare two things you must express them in the same frame. Moving one frame onto another takes three ingredients:

- a **rotation** (mostly a change of *heading*, the compass direction; for a levelled scan the other two angles are small),
- a **translation** (a shift in metres along x, y, z),
- sometimes a **scale** (photogrammetry has no built-in unit; its scale is only as good as the GPS tags, about 1%).

Rotation and translation together are a **rigid transform** (nothing is stretched). It is stored as a 4×4 matrix so that applying it to all points is one multiplication. The pipeline works with such matrices all the time; "registration" is the job of finding the right one.

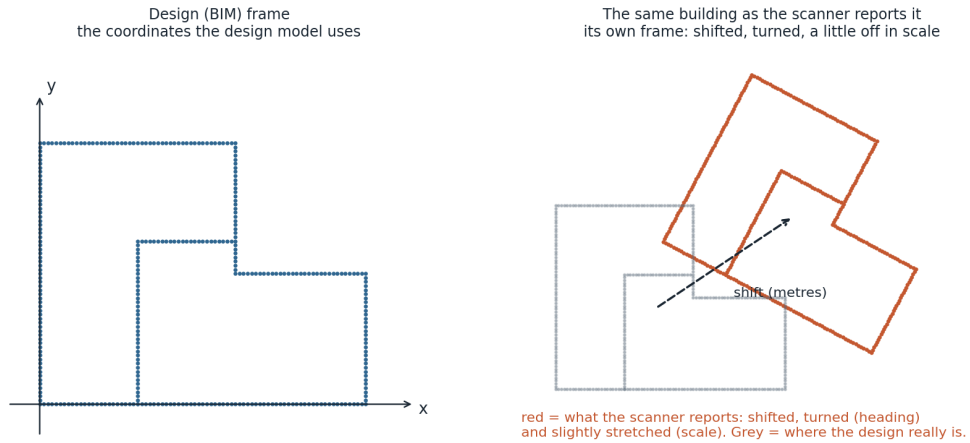


Figure 3. *The same building in two frames. The scanner reports the red outline; the design says the grey one. Registration finds the shift, turn and scale that put red on grey.*

> **In plain words**
> Two maps of the same place drawn with different origins and orientations. Before you can compare them, you have to slide, turn and (sometimes) resize one until the streets line up.

2.5 Registration and ICP

Registration is the problem of finding that transform: given a scan and a model of the same thing, find the rotation, shift and scale that lay the scan exactly on the model. The classic algorithm is **ICP, Iterative Closest Point**. Its idea is simple:

1. For every scan point, find the **closest point** on the model. This gives a pairing.
2. Compute the one rigid transform that makes the paired points as close together as possible (a small least-squares problem with a closed-form answer).
3. Move the scan by that transform. The pairings are now slightly better, so go back to step 1.
4. Stop when the scan no longer moves.

Each round the scan slides a bit closer, like a puzzle piece settling into its hole. Two refinements are used in practice and in our code: **point-to-plane** distance (measure the distance to the model's local surface plane rather than to a single point, which converges faster on flat walls) and a **robust loss** (we use a Tukey loss) that down-weights pairs that are far apart, so clutter and not-yet-built parts cannot drag the answer away.

> **ICP's weakness: it needs a good start**
> ICP only improves what it is given. It looks for the *nearest* model point, and if the scan starts far away or turned, the nearest points are the wrong ones, and the scan settles into a wrong local fit. Figure 4 shows exactly this on a toy floor plan: from a start 12° and about 1 m off it settles in the right place (error 0.01 m); from a start 120° and 6 m off it stops in a wrong place (error 8.7 m) and happily reports that it has converged. On our realistic data, plain ICP was right in **0 of 100** tests.

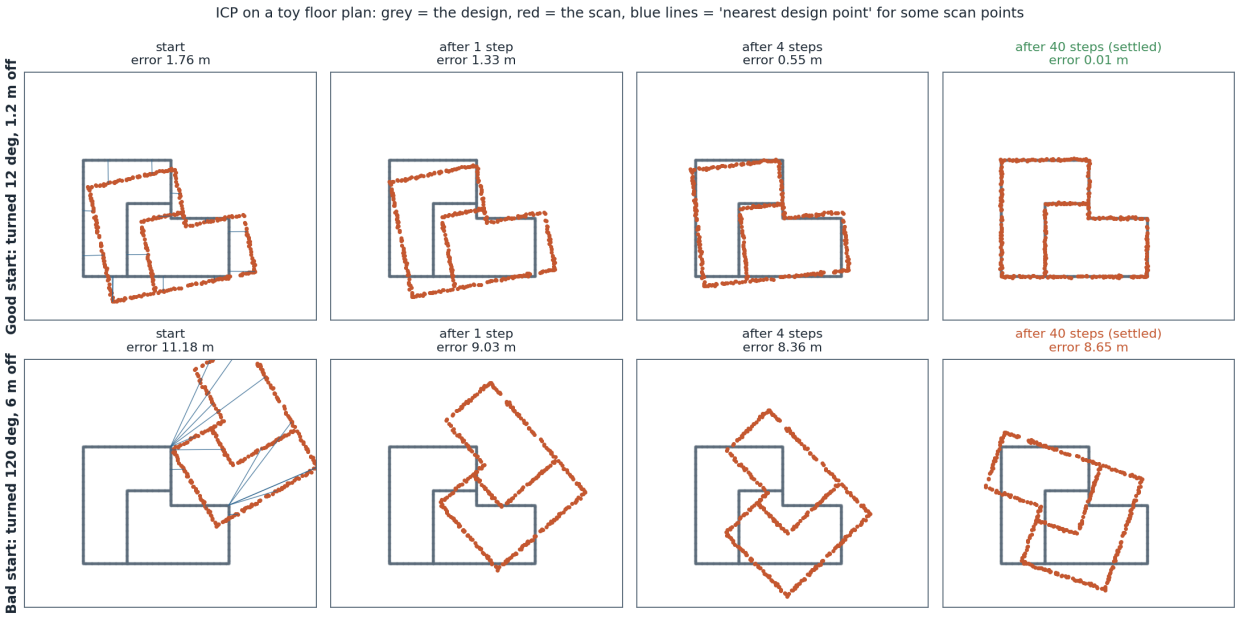


Figure 4. *A real ICP run on a toy floor plan (grey: design; red: scan; blue lines: each sampled scan point's nearest design point). Top: a good start converges. Bottom: a bad start gets stuck.*

2.6 Nearest-neighbour search and KD-trees

ICP, and most of our pipeline, needs the answer to one question millions of times: *which model point is nearest to this scan point?* Our design model is sampled into about 13 million surface points, and a scan has up to half a million points. Checking each scan point against every model point would take 13 million × 500 thousand distance calculations: far too slow.

A **KD-tree** (k-dimensional tree) makes the question fast. To build it, cut the set of points in half along one axis (say x) at the middle value; cut each half along the next axis (y); keep alternating until each cell holds a few points. The cuts form a tree. To find the nearest point to a query, walk down the tree to the cell that contains the query, take the nearest point there as a first guess, and then only look at the other cells that could contain something closer than that guess. Most of the tree is never touched, so each query takes about the time of a few hundred distance calculations instead of 13 million.

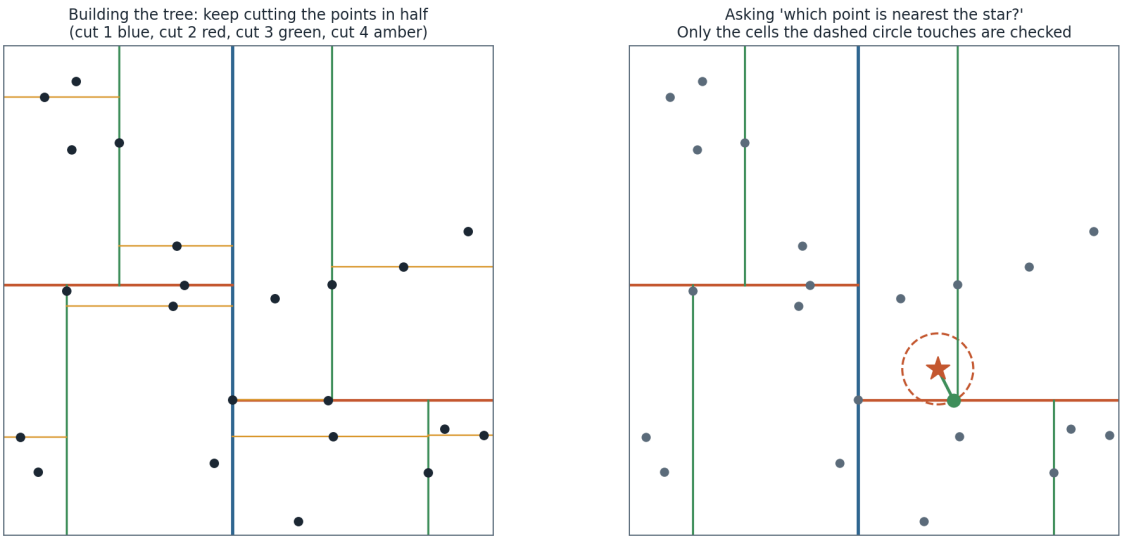


Figure 5. *A KD-tree on 22 random points. Left: the cuts. Right: asking for the nearest point to the star; only the cells touched by the dashed circle need checking.*

> **In plain words**
> Finding the nearest town to you by looking only at the right region of the map instead of reading the whole atlas. We use the SciPy implementation (cKDTree), with the model samples tagged by element ID, so "nearest point" also tells us "which element".

2.7 Finding the rough position: correlation and the FFT

Since ICP cannot find its own start, our registration first searches for a rough alignment by brute force. It turns the problem into one picture comparison. Imagine the top-down floor plan of the walls as a black-and-white image, once for the design and once for the scan. If you slide one image over the other, the number of overlapping wall pixels is highest when they line up. That sliding comparison is called **cross-correlation**. Done naively it is slow, but the **FFT** (Fast Fourier Transform) computes the overlap for *all* shifts at once. We repeat it for every heading (one degree at a time), so the best overlap over all shifts and headings gives a heading, x and y. Height is found separately from the heights of the floor slabs. ICP only has to do the last centimetres.

> **In plain words**
> Lay two transparent floor plans over each other, spin one slowly, and slide it around until the walls overlap the most. The FFT is a shortcut that tries every slide position in one go.

2.8 Ray casting and why we simulate

Ray casting means shooting imaginary rays from a camera position into a 3D model and recording where each ray first hits a surface. Collect the hit positions and you have a point cloud: exactly what an ideal scanner would measure. We use the Open3D library's ray-casting engine on the design model, from drone-like viewpoints around the building and scanner-like viewpoints on each floor.

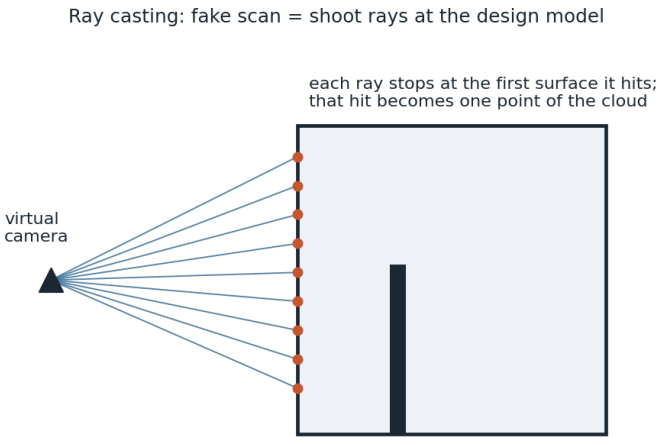


Figure 6. *Ray casting: each ray stops at the first surface it hits, and that hit becomes a point.*

Why simulate? A real scan of a real site is useful, but nobody can tell us *exactly* which elements existed at that moment, so we could not grade our result. In simulation we choose what is built, so the truth is exact; we can also dial in each real-world problem separately and measure its cost. The danger is making the test too easy, so much of the work after the mid-term was making the simulation harsher and more realistic.

2.9 Photogrammetry, SfM and COLMAP

Photogrammetry measures 3D shapes from photographs. The modern pipeline has two parts. **SfM (Structure from Motion)** works out, from the photos alone, where each camera stood and a sparse set of 3D points. **MVS (Multi-View Stereo)** then uses those camera positions to compute a dense depth for almost every pixel. **COLMAP** is the widely used open-source software that does both.

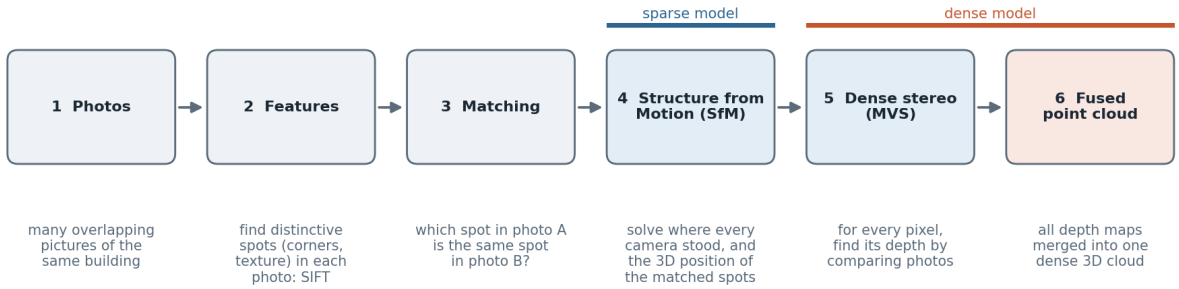


Figure 7. The photogrammetry pipeline COLMAP implements.

SfM step by step

1. **Features.** In each photo, find distinctive spots: corners and patches with texture (the SIFT detector). Each spot gets a numerical description that stays similar when the view changes.
2. **Matching.** Compare descriptions between photos to find the same physical spot in several photos. A geometric check removes wrong matches.
3. **Triangulation.** A spot seen from two known camera positions defines two rays that cross at a 3D point (Figure 8). COLMAP starts with a pair of photos, then adds more photos one at a time: it works out each new camera's position from points already known (*pose estimation*) and triangulates new points.
4. **Bundle adjustment.** After each batch, all camera positions and all 3D points are adjusted together to minimise the **reprojection error**: the pixel distance between where a 3D point should appear in a photo and where its feature really is. A good model has an error below about one pixel; ours is 0.4 to 0.55 pixels.
5. **Dense stereo (MVS).** For each photo, test many possible depths per pixel and keep the one for which neighbouring photos agree (patch-match stereo). This step needs a GPU.
6. **Fusion.** Merge all depth maps into one dense point cloud, dropping points that disagree between views.

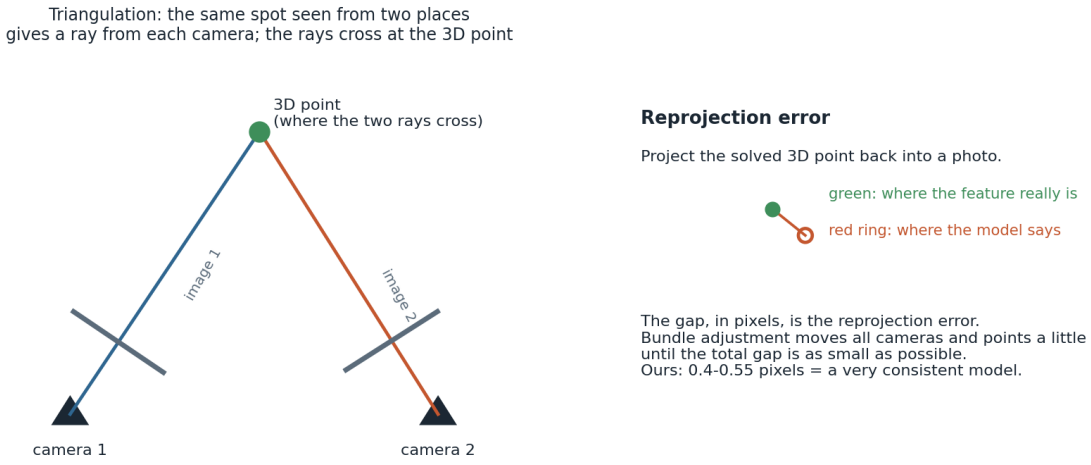


Figure 8. Triangulation and reprojection error.

Two properties that matter for us

Scale and position are arbitrary. Photos alone cannot tell metres from feet or north from south. A real drone records a GPS position in each photo's metadata, and software then fits the solved camera positions to those GPS positions (a scaled rigid transform, or similarity transform). Our experiment does the same with noisy simulated GPS tags, so the COLMAP cloud arrives metres off and slightly mis-scaled, exactly like a real drone survey.

Photogrammetry needs texture. It matches distinctive spots, so plain surfaces, glass, windows and thin objects reconstruct badly or not at all.

2.10 Segmentation, SAM and DART

Segmentation means labelling every pixel (or every point) with what it belongs to, for example "wall", "machine", "sky". **SAM** (Segment Anything) is a family of foundation models that can cut out objects in an image given a prompt such as a text phrase. In this project **DART** is the labelling tool, built on SAM3, that we tried for removing clutter (machines, ladders, barrels) from camera views before they are turned into points. It needs a GPU, so it was run on a different machine.

2.11 How we measure success

- **Ground truth**: the exact answer, from the manifests of the stages we generated. It is used *only* for grading, never by the pipeline itself.
- **Progress** = sum over elements of how complete each is (1 = built, 0 = not started, a fraction for partly built walls), divided by the number of elements. **Count-weighted** treats every element equally; **volume-weighted** weights an element by its volume, so a big slab counts more than a small beam.
- **Progress error** = estimated progress minus true progress, in percentage points. **Signed** error keeps the sign (negative: we underestimate); we mostly report the average of the absolute error over the 16 stages.
- **Missed built**: an element that is built but called not started. **False built**: an element that is not built but called built.
- **Registration success**: the scan ends within 5 cm of its right place on average.

3. The problem, the data and the system

3.1 Goal and scope

Goal. From a scan of a construction site and the building's design model, estimate for every element whether it is built, and the overall progress, with a statement of how sure we are.

Scope. Four structural element types (walls, slabs, beams, columns); one building; a simulated drone and indoor scanner; a progress number per stage. Not in scope: doors and windows, finishing work, formwork and rebar stages (see Chapter 8), and real data.

Why simulation, and not a public data set?

We looked for a data set that has the design model, real scans over time, how each scan lines up with the design, and which elements really were built in each scan. No public set has all four (details in `docs/DATASETS.md`). The Schependomlaan data set has the design and drone clouds, but its weekly as-planned files cannot be linked to the design elements. So we use only its design model and make the construction stages ourselves.

3.2 The test building

Schependomlaan is a Dutch residential block (TU Eindhoven / RAAMAC data set, IFC2X3 exported from Archicad; the design file is committed as `data/ifc/IFC Schependomlaan.ifc`, licence CC BY 4.0). It has six storeys in the model: foundation level, ground floor, first, second and third floor, and the roof. The 1,355 elements with geometry by storey: foundation 133, ground floor 412, first floor 345, second floor 239, third floor 188, roof 38.

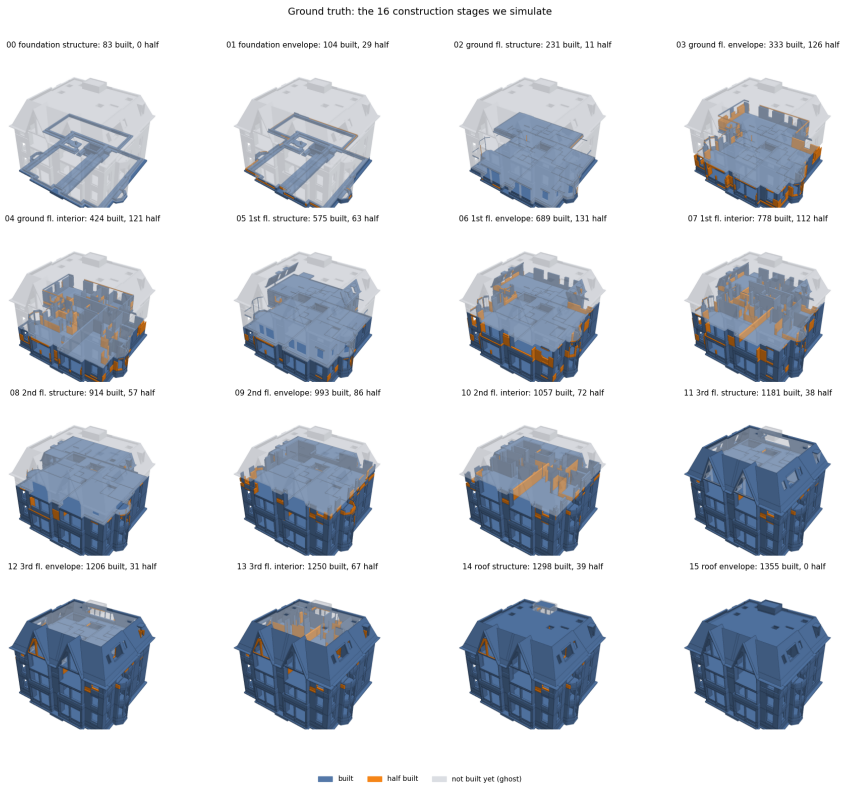


Figure 9. *The 16 simulated construction stages. Blue: built. Orange: partly built. Pale ghost: not yet built.*

3.3 System overview

The system is a chain of separate scripts. Each one reads files from the previous step and writes files for the next; there is no shared memory. This makes every step inspectable (open the CSV or JSON it wrote) and rerunnable on its own. Table 2 lists the steps.

Step	Script	What it does	Writes
P0	p0_sanity_check.py	Counts elements; checks whether the weekly as-planned IDs match the design (they do not)	results/p0_inventory.md
P1	p1_parse.py, materials.py	Reads the IFC into a table (centre, bounding box, volume, storey, interior or exterior) and assigns each element one of 16 material colours	data/bim/bim_complete.csv, bim_materials.csv
P2	p2_ablate.py	Makes the 16 construction stages and the answer key	data/states/*/manifest.json
P3	p3_render.py, p2_p3_simulate_stages.py, site_clutter.py	Simulates the scans (drone orbit and indoor walkthrough) with noise, clutter, offsets, colour	data/pointclouds/*.npz
P4a	registration.py, p4_match_bim.py	Aligns each scan with the design	data/matches/*/registration.json
P4b	p4_match_bim.py	Assigns each scan point to a design element; decides which elements are detected	matches.json
P4c	p4_visibility.py	Would the cameras have seen an element if it were built?	visibility.json
P7	p7_score.py, r5_calibrate.py	Status per element, progress per stage, error bars	data/status/, results/main_table.csv
Experiments	color_registration.py, dart_label.py, sfm_experiment.py	Colour, AI clutter removal, COLMAP	results/experiments/, media/

Table 2. *The pipeline steps, the script behind each, and what each writes.*

A rule that protects the results

Ground truth never reaches the pipeline. The manifests and the `gt_*` fields stored inside the simulated point-cloud files (true element IDs per point, the true alignment) are read only by the grading and diagnostic scripts. Registration, matching and visibility use only what a real survey would have: the points, their colours, the camera positions, and the design model.

4. The pipeline, step by step

4.1 P1: from IFC to a working table

`p1_parse.py` opens the IFC with the IfcOpenShell library. For every wall, column, beam and slab it builds the triangle mesh and records the GlobalId, type, storey, whether the element is external, its centre, bounding box, a plan-view oriented box, its volume and the elevation of its bottom. The result, `data/bim/bim_complete.csv`, has 1,355 rows and is the reference table for everything else. `materials.py` reads each element's material from the IFC's own material list and sorts it into one of 16 colour categories (brick, calcium silicate, concrete, steel, wood, and so on); this is used by the colour experiments.

4.2 P2: the 16 construction stages and the answer key

`p2_ablate.py` turns the complete design into a believable construction sequence. Per storey, from the bottom up, there are up to three milestones:

1. **structure**: columns, beams and the floor slab of that storey,
2. **envelope**: the external walls,
3. **interior**: the interior partition walls.

A storey without a category skips that milestone (the foundation has no interior walls; the roof has none either). This matches the real schedule and real practice: the shell closes before partitions go in. The number of stages (16) falls out of the data.

- **Work front.** Inside a milestone, elements are sorted by their (x, y) position so that building progresses across the floor, and 8% of neighbouring pairs are swapped (with a fixed seed) so the boundary between built and not built is ragged, like a real work front.
- **Partly built elements.** A newly appearing wall or column has a 50% chance of being only partly built: cut at a random height between 25% and 85% of its final height ("poured but not finished"). A partly built element has a 45% chance per later stage to be finished.
- **The manifest.** For each stage, `manifest.json` lists the fully built element IDs (`kept_ids`), the partly built ones with their completion fraction (`partial_ids`), and counts. This is the ground truth for grading.

#	Storey	Phase	Fully built	Partly built
0	-1 fundering	structure	83	0
1	-1 fundering	envelope	104	29
2	00 begane grond	structure	231	11
3	00 begane grond	envelope	333	126
4	00 begane grond	interior	424	121
5	01 eerste verdieping	structure	575	63
6	01 eerste verdieping	envelope	689	131
7	01 eerste verdieping	interior	778	112
8	02 tweede verdieping	structure	914	57
9	02 tweede verdieping	envelope	993	86
10	02 tweede verdieping	interior	1057	72
11	03 derde verdieping	structure	1181	38

#	Storey	Phase	Fully built	Partly built
12	03 derde verdieping	envelope	1206	31
13	03 derde verdieping	interior	1250	67
14	04 dak	structure	1298	39
15	04 dak	envelope	1355	0

Table 3. *The 16 stages (from the manifests). "Partly built" elements are cut to a height.*

4.3 P3: simulating the scans

This is the largest part of the code (`p3_render.py`, with shared helpers in `p2_p3_simulate_stages.py` and the site items in `site_clutter.py`). For each stage it builds a 3D scene of the elements that exist, shoots rays from camera positions, and turns the hits into a point cloud with all the imperfections of a real survey. It replaced the originally planned Blender image renders (SPEC phases P3 and P5 were never built).

Cameras

- **Exterior drone orbit:** 3 altitude rings × 20 positions = 60 viewpoints around the building, each a 70° field-of-view camera pointed at the building (150 × 100 rays each).
- **Interior walkthrough:** on every storey level, a 3 × 3 grid of positions inside the shrunk footprint, each a 360° scan like a tripod laser scanner (110 × 55 ray fan).
- **Ground and trees** are added as fixed clutter that is not in the design.

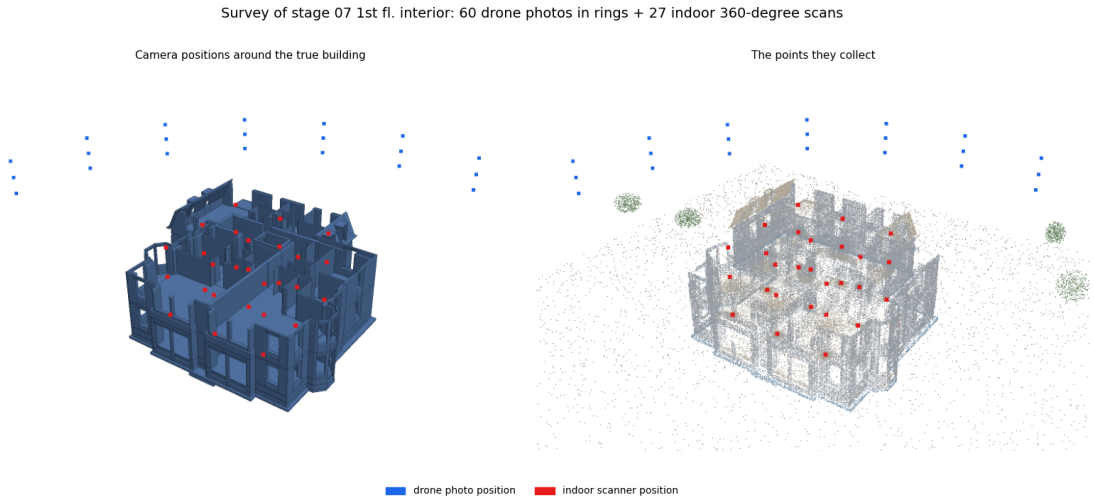


Figure 10. *Where the simulated cameras stand (left) and the points they collect (right).*

Imperfections, from mild to harsh

Effect	Simple (mid-term) setting	Realistic / busy setting (now)
Point noise	1.5 cm everywhere	5 mm + 0.8 mm per metre of range (a drone at 30 m: about 3 cm), worse at slanted views
Dropped points	15% at random	10% base; points on surfaces seen at a grazing angle (cos < 0.3) fade out entirely
Holes	none	One hole (30 cm to 1.5 m) per 60 m² of surface in the drone-photo points only (a laser needs no texture)
Stray points	none	1% 'mixed pixel' ghosts behind edges, 0.3% random floaters
Element placement	all elements 2 cm and 0.3° jittered	same, plus 4% of elements 4-12 cm off, 1.5% of walls and columns 0.6-1.5 m off, 3% two stages late
Whole-scan position	8 cm and 1.2° jitter	GPS-like: heading ±15°, shift about 2 m horizontal and 1 m vertical, scale ±1% ('gnss'); or an indoor frame up to 30 m away at any heading ('local')
Site	ground and trees	adds scaffolding, tower crane, fence, props, material piles, workers, and (experimental) machines on the floors
Colour	colour = element class (ground truth!)	camera colour from materials, sun and shadows, dirt, brick pattern, 6% of elements in a different material

Table 4. *What the simulator adds, and how the mid-term setting differs from the current one.*

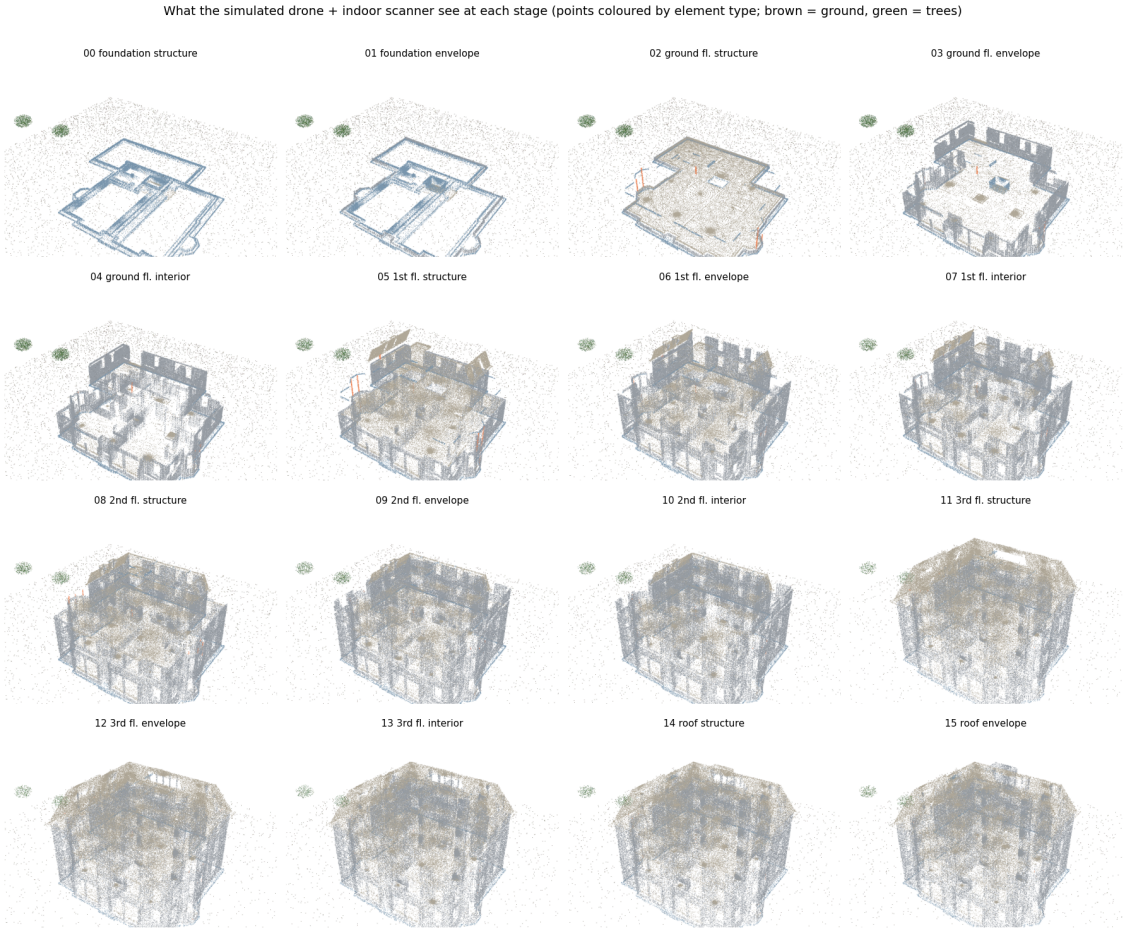


Figure 11. *The simulated scan of every stage (colour shows element type; brown ground, green trees).*

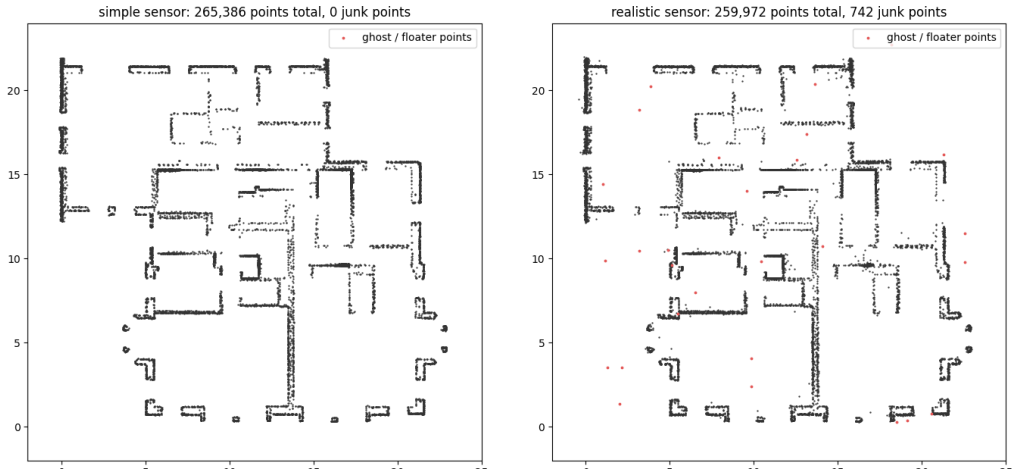


Figure 12. *A slice through the first-floor walls seen from above. Left: simple sensor. Right: realistic sensor with noise, holes and junk points (red).*

The busy site

`site_clutter.py` (option `--site busy`) adds things a real site has and the design does not: scaffolding 0.3-1.0 m outside the facade with plank decks every 2 m (removed at the last stage), a tower crane, a fence around the lot, shoring props under the newest floor, material piles and workers on the ground and floors (new random ones every stage). They are added to the scene as extra objects with no element ID, so they block the view, produce points and carry a clutter label that only the grading may read. The same option applies the as-built errors: elements built a few centimetres off, a few walls about a metre from their place, and a few elements that arrive two stages late. In the grading, late and misplaced elements count as *not built*.

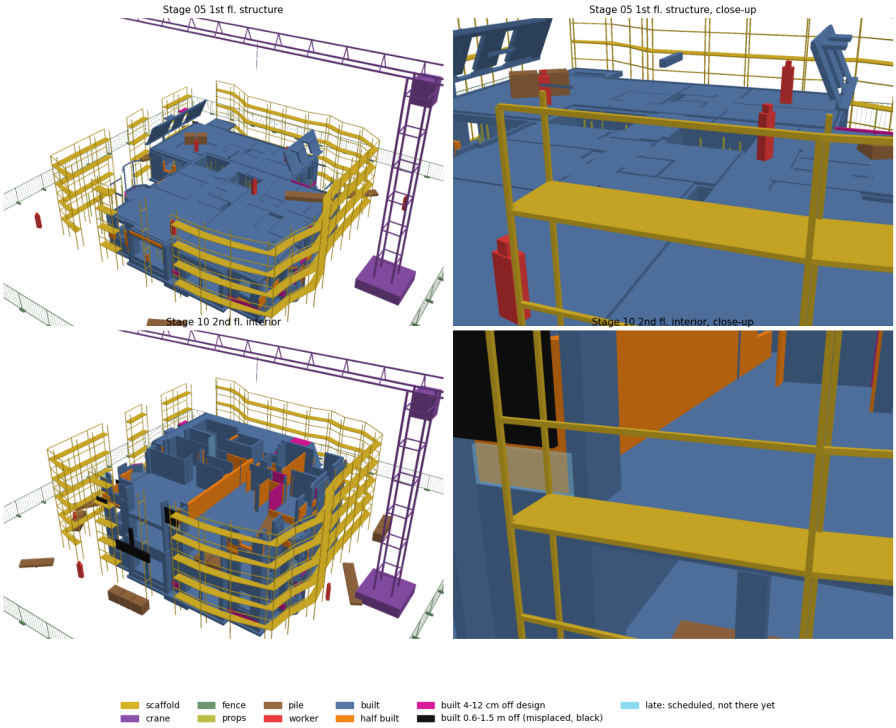


Figure 13. *The busy site: scaffolding (yellow), crane (purple), piles, workers, and elements off the design (pink: a few cm; black: about a metre; light blue ghost: late).*

Camera colour

By default each point also gets a simulated camera colour (`--color material`): the element's material from the IFC is rendered with its own colour table, a per-element tint, a brick or block pattern, a sun with real shadow rays, exposure and white balance per camera, dirt near the ground, and 6% of the elements are rendered in a different material than designed. The matching side knows a deliberately different catalogue of colours, so colour is never a perfect cue. The old class colour is ground truth and must never reach the pipeline.

> **Reproducibility**
> Every random choice uses a fixed seed. The point clouds do not depend on the colour option (colour has its own random streams), and the same options always give the same clouds.

4.4 Registration: laying the scan on the design

This step (`registration.py`, called from `p4_match_bim.py`) was the biggest technical change since the mid-term. At the mid-term the scans started within a few centimetres of the design, so plain ICP worked. A real drone survey is georeferenced by on-board GPS and compass, which leaves it metres off and several degrees turned. In our benchmark, plain ICP succeeded in 0% of such cases, and every failure was silent.

The method: coarse to fine

1. **Level the cloud.** Gravity is known (drone and scanner both have an IMU). Rotate the cloud so that the average normal of its near-horizontal surfaces points straight up. Even 1° of tilt shifts heights by about 0.5 m across a 30 m building, which used to hide the right floor height.
2. **Split the surfaces.** Estimate a normal for each point. Near-vertical surfaces are walls; near-horizontal ones are slabs and ground. Tree canopy falls in neither, so no clutter rule is needed.
3. **Find the height.** Compare the histogram of horizontal-surface heights with the design's. A one-storey cloud fits several storeys, so the top few candidates are kept.
4. **Find heading, x and y.** For each height candidate, draw the design's walls in that height band as a floor-plan picture, draw the cloud's wall points the same way, and use FFT cross-correlation (Section 2.7) for every heading in 1° steps. The best overlap gives heading, x and y.
5. **Height sweep and 3D check.** A dense sweep of heights scores how many points sit near a design surface; the best candidates are tried with a quick coarse ICP and ranked by that score. (This was needed because a foundation-stage cloud is mostly ground, which is horizontal but not in the design, so the 2D picture alone picked the wrong height.)
6. **Refine.** The best three distinct candidates get a multi-scale robust point-to-plane ICP; the winner is the one with the most points within 5 cm of the design. With GPS data a final step also estimates scale.
7. **Flag doubt.** The result carries an `ok` flag. It is switched off if the cloud is tilted more than 2°, the fit is poor, the result is implausible given the GPS prior, or the second-best candidate is nearly as good as the best (a look-alike answer, like a row of identical flats). A flagged result is a warning, not a guess.

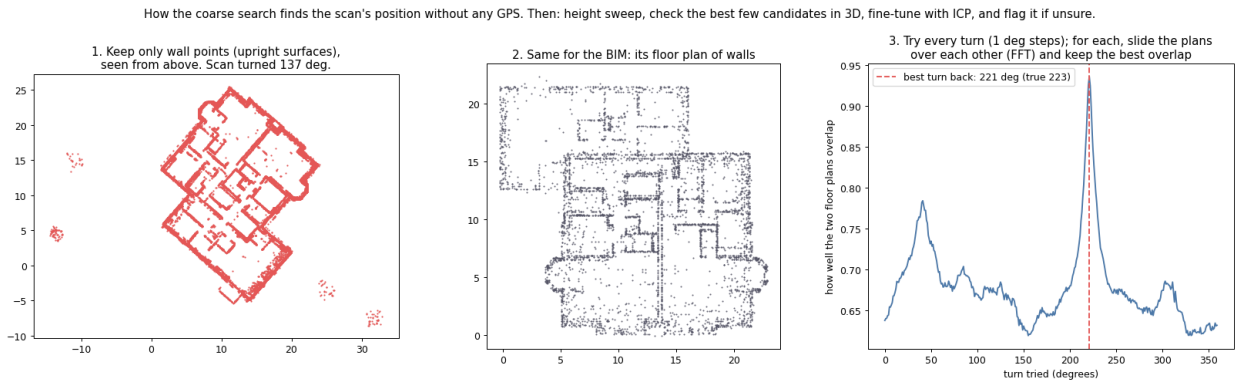


Figure 14. *How the coarse search works: keep wall points, make a floor-plan picture of the scan and of the design, and find the turn and shift with the best overlap.*

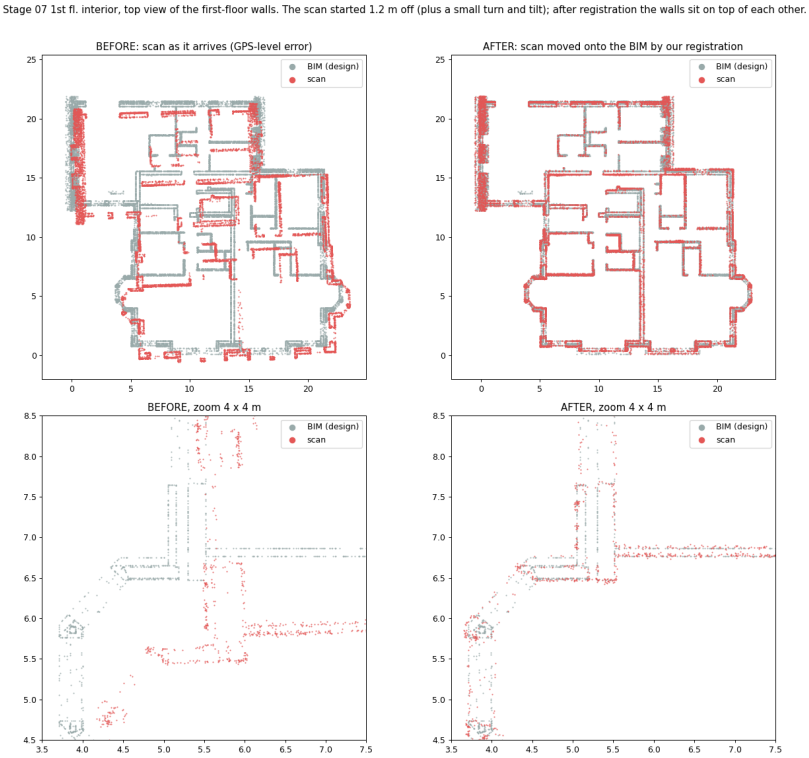


Figure 15. *One stage before and after alignment: the scan (red) starts about a metre off and slightly turned; after registration it sits on the design (grey).*

How well does it work?

Method	Drone with GPS offset	Indoor scan, no GPS	Silent failures
Plain ICP (mid-term)	0% (50 trials)	0% (50 trials)	all of them
Feature matching + RANSAC (FPFH)	40% (10 trials)	30% (10 trials)	6 of 20
Floor-plan search + ICP (new)	98% (50 trials)	98% (50 trials)	1 of 100

Table 5. *Registration benchmark: success means the scan ends within 5 cm of its true place. Stages 00, 03, 07, 11 and 15, random offsets (results/r3_registration_summary*.md).*

In the full pipeline every stage ends up within about 1 to 3 cm. In the harder busy-site runs the maximum was 4.9 cm. Across all registration tests the new method failed in 4 of 170 trials: two were flagged by the method itself, and two were near misses of 7 and 9 cm against the 5 cm pass mark. Feature matching (FPFH) was tried first; on partly built, repetitive buildings it matched wrong places, which is why it was replaced.

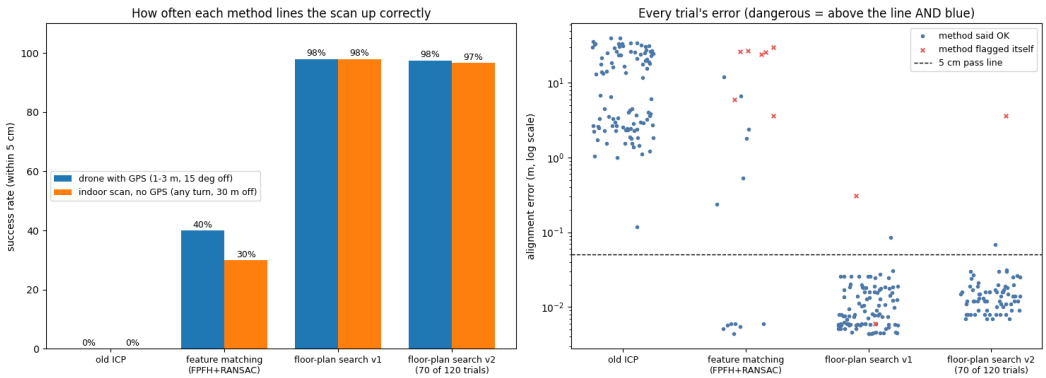


Figure 16. *The registration test bench (left: how often each method lines the scan up correctly; right: every trial's error).*

Using colour in registration

`color_registration.py` (`--registration plan_color`) wraps the method without changing it: it drops points whose colour fits no design material, uses colour agreement only to choose between near-tied candidates, and can polish with Open3D's coloured ICP. Results are in Section 5.1.

4.5 Matching scan points to design elements

After registration the scan sits on the design. We do not try to discover walls in the cloud from scratch (an earlier blind method, `p4_segment.py`, matched very few elements in early stages and is kept only as a baseline). Because the design already says where each element should be, we *match*:

1. **Tag the design.** Sample the whole design surface densely (800 samples per square metre, about 13 million samples) and tag each sample with its element's GlobalId.
2. **Build a KD-tree** over the samples (Section 2.6).
3. **Assign with a margin.** For each scan point, find the 8 nearest samples. The point is given to the nearest sample's element only if it is within **8 cm** and the nearest sample of a *different* element is at least **4 cm** farther away. The margin matters at joints where two elements touch, where noise would otherwise flip points between them.
4. **Detect.** An element with at least 8 assigned points is *detected*.
5. **Read completeness.** The *height coverage* is the height spanned by an element's points divided by the element's true height. Coverage of 0.5 or more means **completed**; between 0.15 and 0.5 means **partly built** (completion = coverage). (This replaces the 3D IoU of the original specification, which makes no sense when a scan only sees surfaces.)

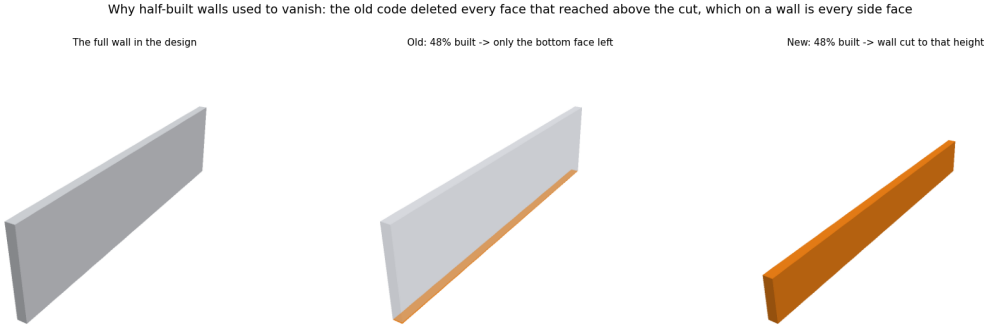


Figure 17. *The half-built wall bug. Left to right: the full wall, the old half-built wall (only a hidden bottom face left), the corrected one (cut to the height built).*

> **A bug that taught us something**
> The simulator used to make a half-built wall by deleting every face that reached above the cut height. That deleted the side faces too, leaving only a hidden bottom face, so the wall almost vanished from the scan and the estimate was always 3 to 8 points too low. Cutting the wall down to the height built fixed it: average error 4.8 → 2.1 points. The lesson: when the result is always biased in one direction, check the simulator, not just the method.

4.6 The visibility check: not built, or just not seen?

If an element has no points, there are two explanations: it is not built, or the cameras could not see it (it is hidden behind other things, too small, pointing away). At the mid-term we treated both as not built. That is wrong, and the old way of excusing it ("elements never detected in the finished building") is circular: at the last stage it can never be wrong. `p4_visibility.py` asks the question directly:

1. Take the actual camera positions of this stage's survey (as a real survey would know them) and the *estimated* registration.
2. Put back the elements we did detect, and cast rays from the cameras.
3. For an undetected element, count how many points it *would* have received if it were built: the rays that reach it and that the matching step could assign to it.
4. If that expected count is at least 30, we should have seen it, so it is **not started**. Otherwise it is **uncertain** and left out of the progress number.

A second use of the same calculation is the **10% rule**: if a *detected* element received fewer than 10% of the points expected for it, the detection is thrown out as noise. With a noisy sensor, a few stray points from a nearby slab land inside the volume of an unbuilt wall and made it look built; this rule cut false "built" calls on the realistic run from 105 to 26.

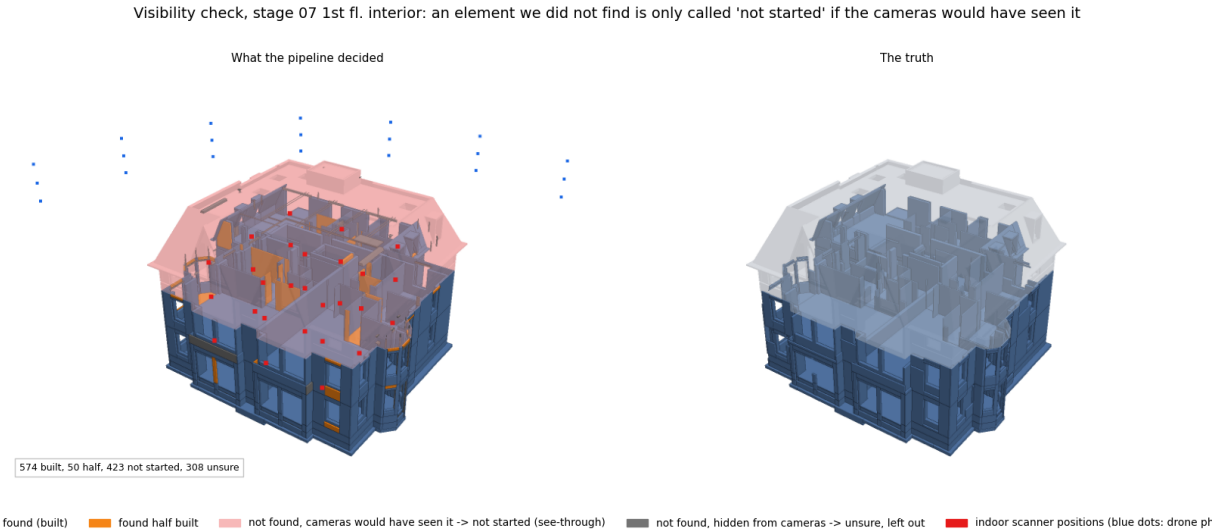


Figure 18. *Visibility at stage 7. Pink and see-through: not started. Dark grey: uncertain (the cameras could not have seen it). The truth is on the right.*

4.7 P7: status per element and the progress number

`p7_score.py` combines the matches and the visibility into one status per element:

Status	Rule	Counts as
completed	detected, height coverage ≥ 0.5	1.0
partial	detected, coverage between 0.15 and 0.5	its coverage
not started	not detected, but would have been seen (expected points ≥ 30)	0
uncertain	not detected, and would not have been seen	left out of both the sum and the count

Table 6. *Element statuses.*

Progress = the sum of the completions divided by the number of non-uncertain elements, times 100, computed both count-weighted and volume-weighted. The true progress is computed from the manifest *over the same set of non-uncertain elements*, so the comparison is fair. The table `results/main_table.csv` has one row per stage with the estimate, the truth, the error, the numbers of missed-built and false-built elements, the registration error, and the rejected-detection counts. On the busy site the late and misplaced elements count as not built in the truth, and extra columns say how many the pipeline wrongly called built.

4.8 R5: error bars on the progress number

A single number invites false confidence. `r5_calibrate.py` and `p7_score.py` add a bias-corrected progress and a 90% range:

1. Every element is put in an **evidence group**: its status; whether its storey is below, at or above the *construction front* (the highest storey with at least 10 completed elements, found without the truth); its type (wall, slab, beam, column); and how many points it got compared with how many were expected.
2. On one **calibration run** (where the truth is known) we measure for each group the average gap between the pipeline's value and the truth (the *bias*) and its spread. Groups with fewer than 20 members fall back to coarser groups.
3. p7 shifts each element's value by its group's bias and adds up the spreads. The errors inside one stage share causes, so a whole-stage term, measured by leaving each stage out in turn, is added.
4. The calibration run and the test run are **different conditions**: we calibrate on the clean realistic run and test on the busy site, so the ranges are not flattered.

> **Why the construction front matters**
> Without the position in the group, the correction made things worse (1.76 → 2.83 points). A "not started" element below the construction front is wrong 98% of the time (it is almost always a built element that was hidden), while above the front it is never wrong. One average per group cannot know that.

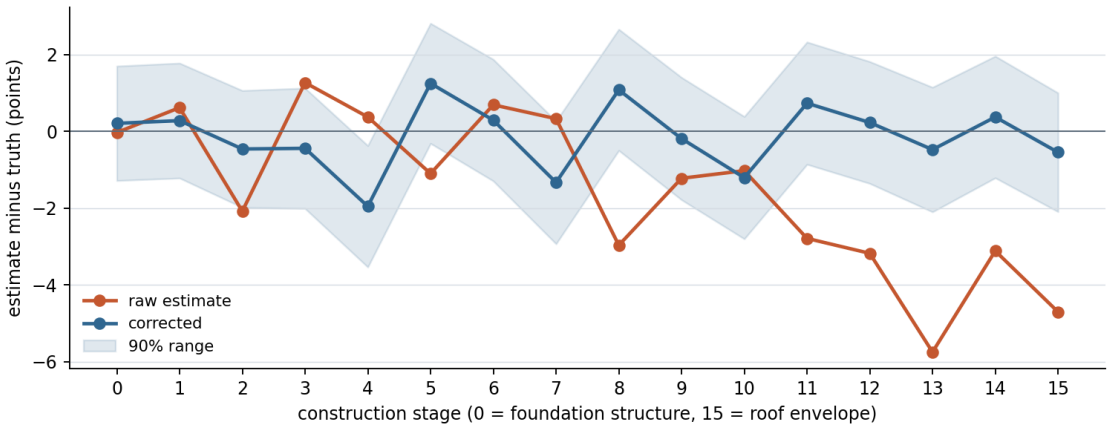


Figure 19. *Estimate minus truth for each stage on the hardest test. The raw estimate drifts down to about -6 points as the building fills up; the corrected estimate stays near zero, and the shaded 90% range contains the truth at 15 of 16 stages.*

Caveats: for volume-weighted progress the correction helps much less and the range is wide (about ±5.6 points) because a few large elements dominate; and the correction holds only for the setup it was learned in (with the 10% rule off, 7 of 16 volume ranges missed the truth).

5. Experiments: colour, AI clutter removal and COLMAP

After the main pipeline was in place we tried three ways to get better information. None replaced the main method, but all three are genuine, measured efforts and each produced a clear lesson.

5.1 Colour

Question. Camera colour changes with light and exposure, but materials still look different (brick is red, steel primer is dark red, insulation is yellow). Can colour help alignment and clutter removal?

What we did. Every point got a simulated camera colour (Section 4.3). The matching side was given a material catalogue with colours that deliberately differ from the renderer's. `plan_color` (1) drops points whose colour fits no material in the design (it drops about half of the tree points and 40% of the crane points and no building points), (2) ranks near-tied alignment candidates by the colour agreement with the material of the design surface they land on (using chromaticity, so brightness matters little), and (3) optionally polishes with coloured ICP.

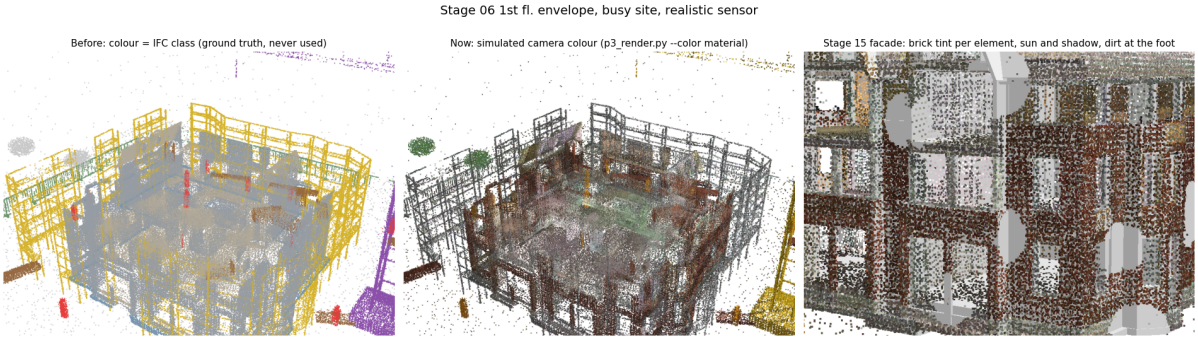


Figure 20. Scan colour before (element type, which is ground truth and never used) and now (simulated camera colour).

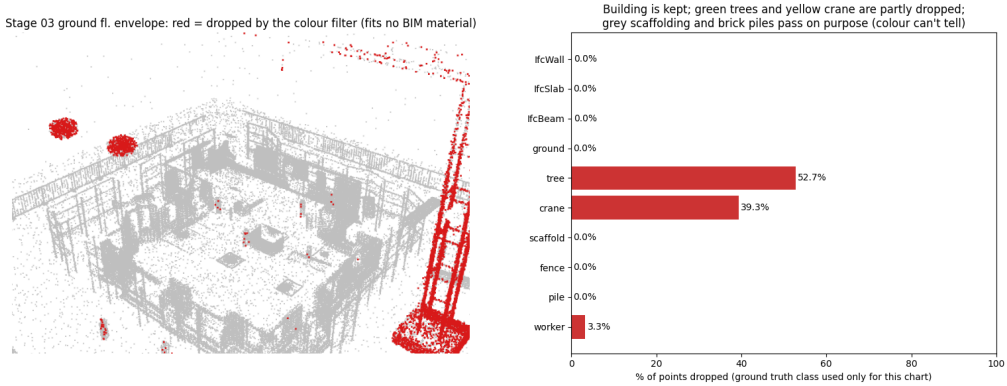


Figure 21. Points dropped by the colour filter (red): about half of the tree points and 40% of the crane, no building points. Grey scaffold and brick piles pass on purpose.

Result. In the registration benchmark (6 stages, two cases, busy and realistic clouds) `plan`, `plan_color_rank` and a version given a deliberately *wrong* catalogue were all right in 12 of 12 cases, with median errors of 1.2 to 1.5 cm and no silent failures. Colour never actually decided: in every run the second-best pose was already more than 10% worse on geometry. In the full pipeline `plan_color` gave an average error of 2.1 points against 2.0 without colour.

Why it does not help. Under varying light, nine of the materials end up in the same grey area of colour space, so colour cannot tell them apart; brick piles look exactly like brick walls. A pose check showed colour does penalise a pose one storey too high (brick lands on grey floors), but not a 3 m sideways shift

or a 180° turn, because the facade is brick all round. Those cases are decided by geometry.

> **A failure worth remembering**
> Our first version let colour overrule geometry. With the deliberately wrong catalogue it put stage 00 3.3 m off and did **not** flag it. The fix: colour may only break near-ties. Lesson: a weak cue must never be allowed to override a strong one without a safety check.

Decision. Geometry stays the default. A natural next step is a model that recognises the material from a texture patch (rather than from raw colour) and compares it with the material the design says the matched element should have (Chapter 8).

5.2 AI clutter removal with DART

Question. Clutter (machines, ladders, scaffold) pollutes the matching. Can an image segmentation model find clutter in camera images so that its points are removed before matching?

What we did (one stage only: stage 11, third-floor structure). We added multi-part machines and tools on the built floors (generator, mixer, compressor, scissor lift, ladder, trestle, barrel, cable reel, toolbox, buckets) and saved simulated camera images that carry, for every pixel, its 3D position. DART (built on SAM3) labels each image; points on pixels labelled clutter are removed from the cloud. DART needs a GPU, so it ran on another machine (`src/dart_label.py`).

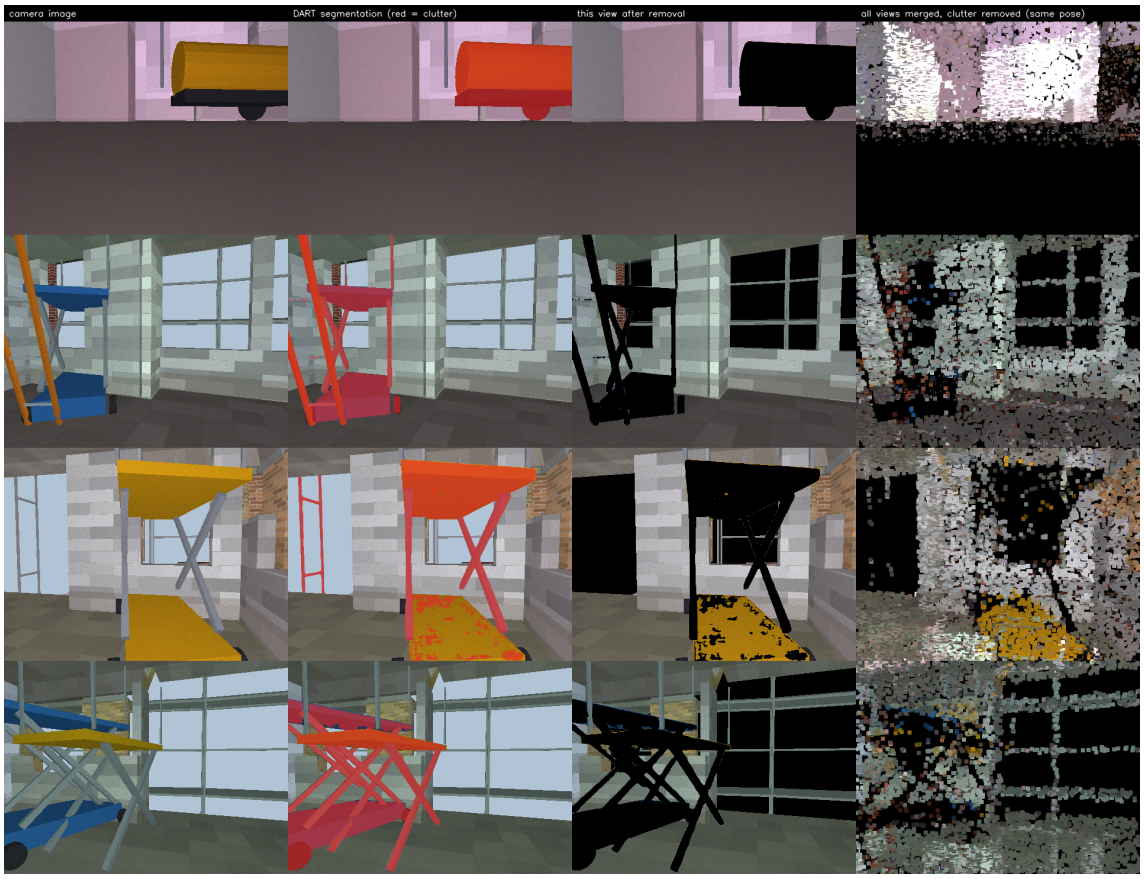


Figure 22. Columns: camera image, DART's labels (red = clutter), the view after removal, and the merged cloud from the same pose.

Result. DART's default confidence (0.6) found none of the plain-box machines. With a lower threshold (0.25) and equipment prompts it removed **41%** of the true clutter points while keeping **99.6%** of the structure. A first attempt that used prompts for structure kept only 68% of the structure and the progress estimate fell to 28% (truth 85%); using "scaffolding" as a prompt swallowed the whole facade. With the good prompts the progress estimate was unchanged, about 48% with or without removal against 85%

true. The reason is not DART: a cloud made only from camera images covers far less of the building than the scanner cloud, so the limit is coverage.

Not done. Using the real camera positions in the visibility check so unseen regions drop out of the count, and running the other 15 stages.

5.3 COLMAP: point clouds from drone photos

Question. All our clouds so far came straight from ray casting. A real drone survey gives *photos*. What happens if the cloud is built by COLMAP from simulated photos, and does it fit into our pipeline?

What we did

1. **Rendered 60 drone photos** of a stage (`src/sfm_experiment.py`): 1280 × 960 pixels, 70° field of view, three altitude rings of 20 positions with a little pose noise. Each photo has material colours, brick pattern, a fixed sun with shadows, a textured ground, per-photo exposure, a slight blur and sensor noise.
2. **Added fine surface detail.** The first lesson: COLMAP finds nothing to match on flat colours. We added detail at three scales (6, 15 and 40 cm) that is fixed on the wall (so it looks the same from every camera, like real brick or concrete grain).
3. **Ran COLMAP** (the pip package `pycolmap-cuda12`): features, exhaustive matching, incremental SfM, then dense stereo on the GPU at 1024 px and fusion.
4. **Moved the cloud into a survey frame.** COLMAP's scale and position are arbitrary (its scale was about 9 times off metres). As a real survey does, we fit the solved camera positions to noisy GPS tags (1.5 m noise) with a scaled rigid fit; the cameras then end up about 0.4 m off.
5. **Ran the cloud through our unchanged pipeline** (registration, matching, visibility, scoring) using the run tag mechanism, and compared with a **perfect cloud from the same photos**: the exact 3D position of every 4th pixel, i.e. what a flawless reconstruction of these photos would give.

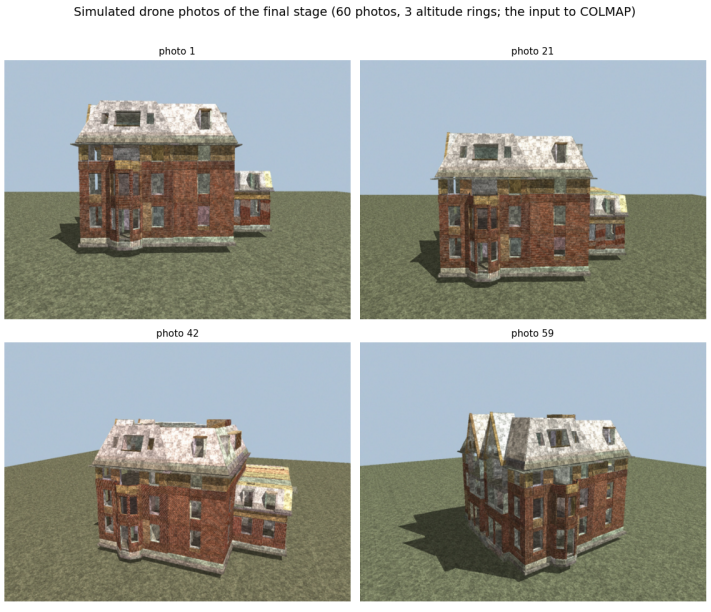


Figure 23. *Four of the 60 simulated drone photos used as input to COLMAP.*

Results

Measure	Stage 07 (first-floor interior)	Stage 15 (finished roof)
Photos registered by COLMAP	60 of 60	60 of 60
Mean reprojection error	0.39 px	0.55 px
Dense cloud size	461,946 points	487,106 points
Median distance to the true surface (aligned with the true cameras)	2.6 cm	3.7 cm
Points within 10 cm of the true surface	72%	73%
Visible surface reconstructed (within 10 cm)	53%	45%

Table 7. *Quality of the COLMAP cloud itself (results/sfm/*_metrics.json). Dense stereo took about 28 minutes per stage on the RTX 3050.*

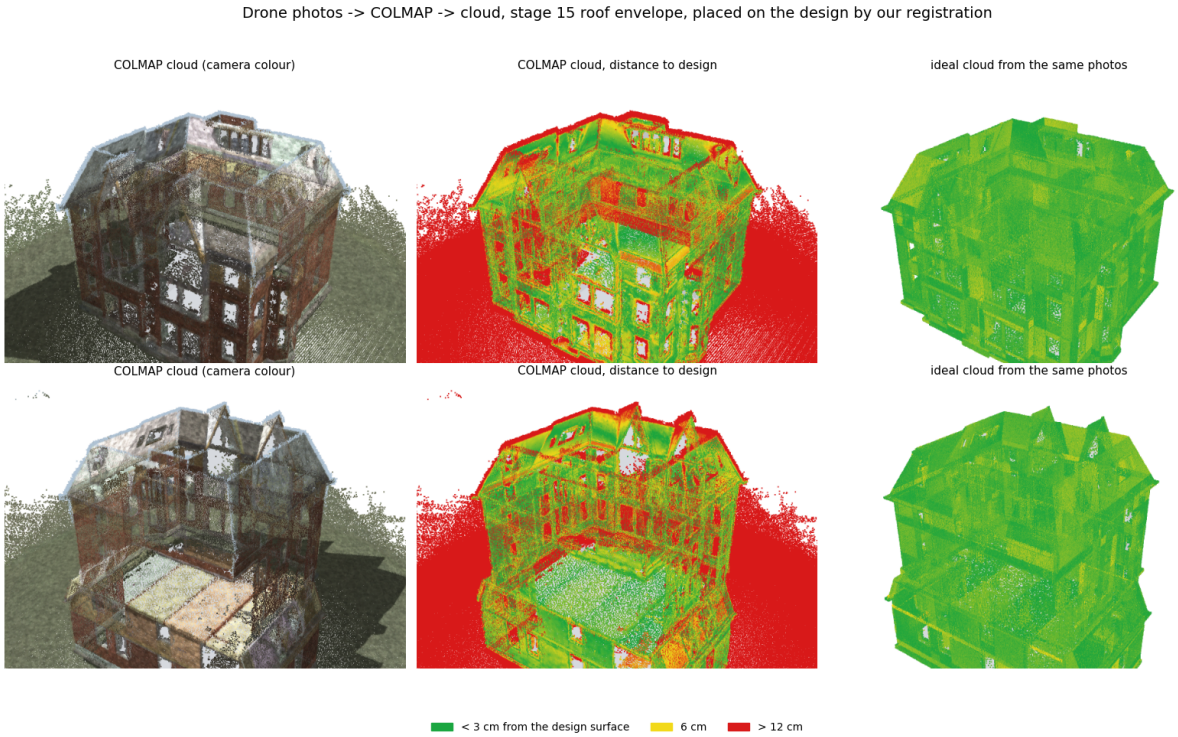


Figure 24. *Left: the COLMAP cloud in camera colour. Middle: the same cloud coloured by distance to the design (green: close). Right: the perfect cloud from the same photos. The ground is red only because it is not part of the design.*

Cloud source	Stage 07 estimate	Stage 15 estimate
True progress	56.4%	100.0%
Ray casting, drone + indoor scans	55.8%	94.4%
Perfect cloud from the same 60 photos	57.5%	94.0%
COLMAP cloud	51.0%	64.7%

Table 8. *The same pipeline on four kinds of cloud (results/experiments/main_table_sfm*.csv).*

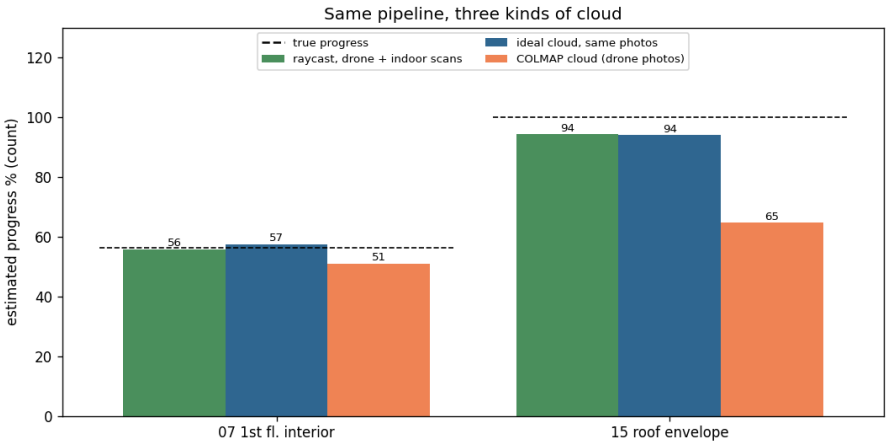


Figure 25. *Estimated progress for the two stages and the four cloud sources.*

What it means

- **The drone viewpoints are good enough.** The perfect cloud from the same photos gives 94% at the finished stage against 100% true, with 2 elements missed.
- **COLMAP loses the rest.** Mid-construction it is 5 points low (51.0% against 56.4%); at the finished building it is 35 points low (64.7% against 100%) with 263 built elements missed. The reconstruction leaves holes at windows, puts a halo of stray points at roof edges and loses thin parts. A looser matching distance (15 cm instead of 8 cm) only moved it from 64.7% to 66.3%, so the main cause is missing coverage.
- **Registration works on COLMAP clouds.** Our alignment placed both clouds correctly; the pipeline's alignment fits the design surface *better* than the alignment made from camera positions alone (49% against 46% of points within 8 cm), so the 0.2 m 'registration error' reported against the camera-based reference mostly reflects that the reference is imperfect.
- **Open question.** Why the finished building does so much worse than the mid stage is not yet understood.

> **Honest status**
> This is a one-and-two-stage experiment, not a full 16-stage run. It shows that a photogrammetry cloud can be fed into the pipeline unchanged and what limits it. Next steps: more and closer photos, oblique flight paths, cleaning stray points, and a different dense stage (for example OpenMVS).

Interior photos: the same experiment inside the building

Everything above used *exterior* drone photos. Indoors the problem is different, so we repeated the experiment on the ground floor of stage 07 (`sfm_experiment.py --views interior`): 18 standing positions on a grid, four views each, 72 photos with a 90° field of view, rendered the same way as the drone photos. There is no GPS indoors, so the cloud was given the metric scale and levelling a handheld rig would provide and an arbitrary start frame, and registration was run without a GPS prior.

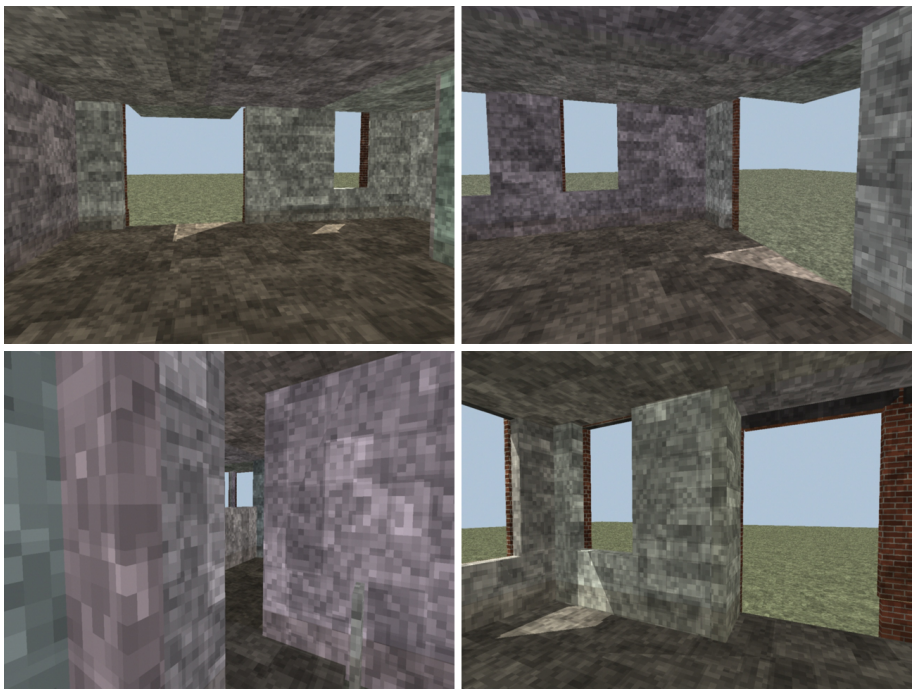


Figure 26. *Four of the 72 simulated interior photos (ground floor, stage 07). Note the openings: the walls have no windows glazed yet.*

Measure	Interior photos
Photos registered by COLMAP (best of 2 models)	26 of 72
Dense cloud size	207,603 points
Median distance to the true surface (where the cloud exists)	4.6 mm
Points within 10 cm of the true surface	99.6%
Registration of the COLMAP cloud (error, and did it warn?)	11.75 m off, **no warning**
Progress from the COLMAP cloud / true progress	9.7% / 69.5%
Progress from perfect 3D of the same photos / true progress	87.9% / 97.3% (registration 2 cm off)

Table 9. *Interior COLMAP experiment (results/experiments/main_table_sfm_int*.csv, results/sfm/*_int_metrics.json).*

- **The photos do not link up.** COLMAP produced two separate models and the larger one used only 26 of the 72 photos. Rooms separated by walls share almost no views, so there is nothing to connect the photos of one room to those of the next. Real indoor capture avoids this with a continuous walk through the doorways; a grid of standing positions does not.
- **Where it exists, the cloud is excellent:** median error 4.6 mm, 99.6% of points within 10 cm. Indoors, with short distances and plenty of texture, photogrammetry is very accurate.
- **But it covers only a few rooms, and that broke registration silently.** A cloud of a few look-alike rooms fits several places in the building. The floor-plan search chose a wrong one, 11.75 m away, and the result still passed our own checks (the look-alike ratio was 0.85, just under the 0.9 flag threshold). The progress estimate from it is meaningless (9.7% against 69.5% true).
- **Even a perfect cloud from these photos sees only part of the building:** it gives 87.9% against 97.3% true, with most elements left uncertain because only the ground-floor interior is visible.

> **What this adds to the limits**
> Our registration flags look-alike answers, but a small partial cloud can slip past the threshold. Indoor photogrammetry also needs a capture plan that keeps the photos connected. Both are open items (Chapter 8).

6. Results and how we got here

6.1 What each change did

Figure 27 puts the main results together. The top block takes the hardest world (realistic sensor, busy site, late and misplaced elements) and removes one change at a time; the middle block shows the same pipeline as the world gets harder; the bottom block shows how the pipeline improved on the easy world.

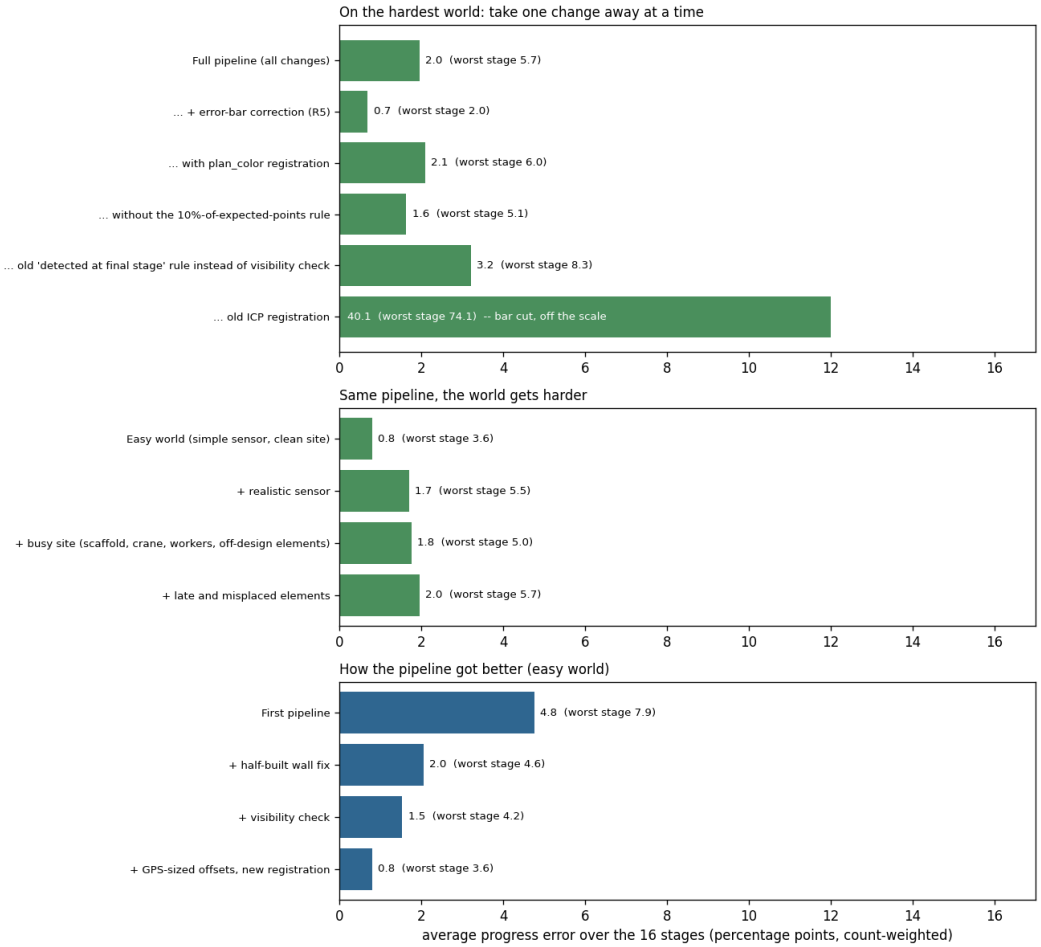


Figure 27. *Average progress error over the 16 stages, in points (lower is better).*

Run	Mean error	Worst stage	Volume error	Missed built	False built
Full pipeline (all changes)	1.95	5.75	2.24	327	68
+ error-bar correction (R5)	0.69	1.95	2.08		
with plan_color registration	2.09	6.04	2.08	351	64
without the 10%-of-expected-points rule	1.63	5.12	6.39	263	160
old 'detected at final stage' rule instead of visibility check	3.21	8.28	5.70	658	160
old ICP registration	40.09	74.10	30.56	6921	382
Easy world (simple sensor, clean site)	0.79	3.64	2.07	98	50
+ realistic sensor	1.69	5.49	2.17	276	32
+ busy site (scaffold, crane, workers, off-design elements)	1.76	5.00	2.10	308	67

Run	Mean error	Worst stage	Volume error	Missed built	False built
+ late and misplaced elements	1.95	5.75	2.24	327	68
First pipeline	4.76	7.89	2.82		
+ half-built wall fix	2.05	4.60	1.94	387	45
+ visibility check	1.53	4.23	2.02	276	45
+ GPS-sized offsets, new registration	0.79	3.64	2.07	98	50

Table 10. *The improvement ladder (results/improvement_ladder.csv). Group order: hardest world, world getting harder, history.*

- **The new alignment is essential.** Old ICP on the hardest clouds: 40 points of error, the scan about 3 m off, no warnings. New: 2.0.
- **The visibility check roughly halves the misses.** The old final-stage rule gives 3.2 points and 658 missed built elements against 327 now.
- **The 10% rule trades count for volume.** Without it the count error is slightly lower (1.6) but the volume error nearly triples (6.4 against 2.2) and false built calls go from 68 to 160.
- **Colour registration did not help** (2.1 against 2.0).
- **The world getting harder costs about 1 point**: 0.8 (simple sensor) → 1.7 (realistic sensor) → 1.8 (busy site) → 2.0 (late and misplaced elements).

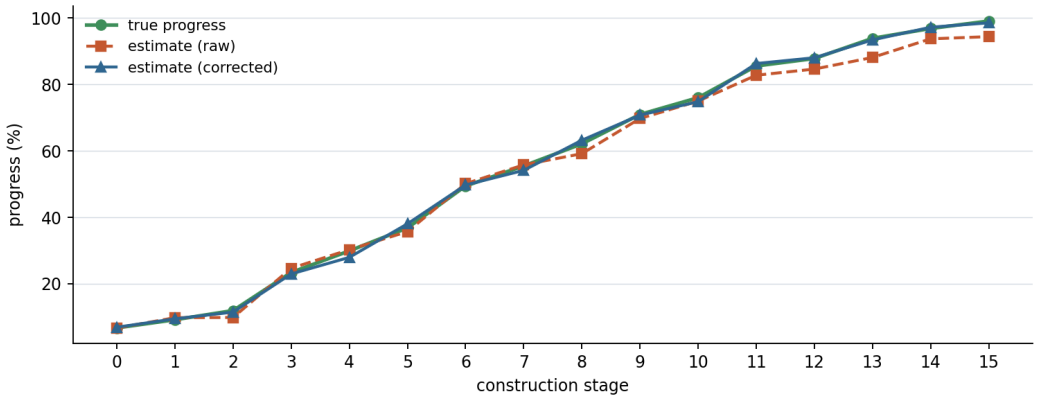


Figure 28. *Estimated and true progress for the 16 stages on the hardest test.*

6.2 Timeline of the work

Date	What happened and why it mattered
2026-09-18	First pipeline: parse the design, 16 stages, ray-cast scans, ICP, matching, scoring (the mid-term state). Mean error 4.8 points; the estimate always ran low.
2026-10-27	Found the half-built wall bug (side faces dropped). Added the per-stage visibility test. Started the new registration. Mean error 4.8 → 1.5.
2026-10-27/28	New registration: gravity levelling, floor-plan FFT search, height sweep, 3D verification, ambiguity flag. Benchmarked: old ICP 0%, FPFH 30-40%, new 98%. Stage 01 registered 0.6 m too high without a flag; fixed by keeping the top three height peaks and ranking by tight fit. Mean error 0.8.
2026-10-29	Realistic sensor run: error rose to 1.3. Found and fixed two simulator bugs (holes erasing rooms; holes also hitting laser scans) and one pipeline weakness (stray points making unbuilt walls look built), leading to the 10% rule. Added media folder, busy site, late and misplaced elements, colour model, and the R4 threshold check.
2026-10-29/30	Error bars (R5). Colour registration benchmark (12 of 12 right; colour never decided). Full pipeline with colour: no gain. Design IFC committed so a fresh clone can run P1-P7.
2026-10-31	Interior machines and DART clutter removal tried on one stage (Sagar).
2026-11-01	COLMAP experiment on two stages. Old ICP run on the hard clouds (40 points). Improvement ladder. End-term slides and this report.

Table 11. *Timeline (docs/PROGRESS.md has the full dated log).*

6.3 Things we tried that did not work

- **Cleaning the cloud** with outlier removal and plane smoothing: it also removed good sparse points from far beams, so more built elements were missed.
- **A surface-direction (normal) check** against each element's normal: false built 25 → 5 on one stage, but missed built 24 → 41.
- **A noise-adaptive matching distance:** no effect.
- **A sturdier height measure** (trim the top and bottom 2% of points, or count only 10 cm bands with several points): fixed single large elements but made the count error worse (1.8 → 2.2 to 6.9), because many truly built walls have few points.
- **"A detected slab or beam is always complete":** count error improved but volume error got worse.
- **Feature-matching (FPFH) registration:** picks wrong matches on partly built, repetitive clouds.
- **Letting colour overrule geometry in registration:** put a stage 3.3 m off, unflagged.

6.4 Found by looking carefully

- Two bugs in *our own simulator* made the realistic sensor look worse than it was: texture-less holes were centred on random rays and erased whole rooms, and they also hit the indoor laser scans (a laser does not need texture). Both fixed.
- Of the 67 false "built" calls on the busy site, 28 came from clutter: 13 from scaffold plank decks matched as unbuilt balcony slabs, 9 from workers and 6 from material piles standing where unbuilt walls would be.
- The late and misplaced elements fooled the pipeline rarely: 3 of 78 late cases and 7 of 108 misplaced cases were wrongly called built (all 7 misplaced were tiny wall pieces).
- The visibility threshold of 30 expected points is a little low: raising it to 80 cuts missed built elements and the worst-stage error on both the simple and realistic runs, at the cost of about 4% more uncertain elements. It was not changed because it would be tuned on the same runs it is tested on.

7. Limitations and honest caveats

- **Simulation only, one building.** No real survey has been processed. Simulated noise is modelled on what we know of real sensors, but real effects (glass, moving workers, weather, rolling shutter, lens distortion, georeferencing drift) may behave differently.
- **Tuned on the data it is tested on.** Thresholds (assignment distance, the 10% rule, the visibility threshold) were chosen while looking at these runs. A frozen test split is still needed (roadmap R0).
- **About 25% of elements are never seen** by any camera plan even when built (tiny pieces, hidden slabs). They are reported as uncertain, and excluded from the count, which makes the progress number cover roughly three quarters of the elements.
- **The error-bar correction is tied to its setup.** It was learned on one run type; its volume-weighted range is wide.
- **Clutter still fools matching** in places (scaffold decks, workers, piles).
- **Registration can fail silently on small partial clouds.** A cloud of a few look-alike rooms (our interior COLMAP test) was placed 11.75 m off without a warning; indoor photo capture also fragmented (26 of 72 photos linked).
- **The indoor scanner height was not checked.** The simulator's level merge puts the ground-floor indoor scanners at floor level (z = 0.0) instead of 1.4 m above it; we noticed it late and have not measured its effect.
- **Only built or not built, plus a height cut.** Real progress has activities (formwork, rebar, concrete, finishing) that we do not model.
- **The history rows are not directly comparable with the hard-world rows.** The steps from 4.8 to 0.8 points were measured on the easy simulated world; later rows use harsher clouds. The 'first pipeline' is the code of the first commit, which we take as the mid-term state.

8. What is left, and what we would do next

Direction	Why	First step
Real data check	The most important unknown: which simulated effects really matter?	Run the pipeline unchanged on a real survey with a known design (candidates: ConSLAM, SLAM2REF samples, Schependomlaan clouds); see docs/DATASETS.md
Frozen test split	Without it, tuned thresholds look better than they are	Freeze which stages and conditions are for testing; choose thresholds on the rest (roadmap R0)
Drone-only flights	A real drone cannot scan indoors; our best results use indoor scans	Exterior-only runs with oblique and closer views; measure how much a drone alone can see
Better COLMAP clouds	The finished building is 35 points off; indoors the photos do not link up	More and closer photos, a continuous indoor walk, cleaning stray points, OpenMVS, then all 16 stages
Safer registration of partial clouds	An indoor COLMAP cloud was placed 11.75 m off without a warning	A stricter look-alike flag and a second independent check for small clouds
Fix clutter false-built	28 of 67 false calls came from clutter	Require a detection to appear again in the next scan (workers and piles move, scaffolding comes down)
Material recognition	Raw colour is a weak cue; texture is stronger	A model that reads material from a texture patch and compares it with the design's material
Merge two scans	Two runs of the same building need aligning with each other; colour should help more here	Use colour registration between scans of the same building
Work activities	Progress is more than built or not built	Formwork, rebar, concrete states in the simulator and the scoring
DART on all stages	Only one stage tested	Needs a GPU machine; add camera-based visibility

Table 12. *Open directions (also in TODO.md and docs/ROADMAP.md).*

9. Using the repository

9.1 Where things are

Folder	Contents
src/	All code: one script per phase (p0 to p7), registration.py, color_registration.py, site_clutter.py, sfm_experiment.py, dart_label.py, r3_registration_bench.py, r5_calibrate.py, and the make_* scripts for media, slides and this report
data/ifc/	The design IFC (49 MB, committed, CC BY 4.0)
data/bim/	Working tables: bim_complete.csv (1,355 elements), bim_materials.csv; cached surface samples (not in git)
data/states/	16 stage folders with manifest.json (the answer key)
data/pointclouds*/	Simulated clouds (not in git). Tagged folders such as pointclouds_sfm hold experiment clouds
data/matches*/, data/status*/	Per-stage registration, matches, visibility and per-element status; the suffix is the RUN_TAG of an experiment
data/sfm/, data/dart/	Photos and COLMAP files; DART images (large, not in git)
results/	main_table.csv, results/experiments/ (every before/after table), improvement_ladder.csv, r5_calibration.csv, benchmark summaries, report and presentation
media/	Pictures, animations and 3D files of the whole project, with an index in media/README.md
docs/	SPEC.md (original specification), ROADMAP.md (plan R0-R7), PROGRESS.md (dated log), DATASETS.md
TODO.md, CLAUDE.md	The checklist, and the guide to the code for AI assistants and newcomers

Table 13. *Folder map.*

9.2 How to run it

Everything is run from the repository root with the project's virtual environment. Install the dependencies listed in CLAUDE.md (ifcopenshell, numpy, scipy, pandas, matplotlib, open3d, scikit-learn, python-pptx, and for COLMAP `pycolmap-cuda12`). The order is the phase order:

```
python src/p1_parse.py # IFC -> data/bim/bim_complete.csv
python src/materials.py # materials -> bim_materials.csv
python src/p2_ablate.py # -> data/states/*/manifest.json
python src/p3_render.py --sensor realistic --georef gnss --site busy
python src/p4_match_bim.py # registration + matching
python src/p4_visibility.py # needs registration.json
python src/p7_score.py # -> data/status/, results/main_table.csv
python src/r5_calibrate.py # error-bar calibration (optional)
python src/sfm_experiment.py --stage 15_04_dak_envelope --step all # COLMAP experiment
python src/make_media.py ; python src/make_endterm_presentation.py ; python
src/make_report.py
```

- P3 and P4 take minutes per stage and about 4 GB of memory on an 11 GB machine: do not run two P4-sized jobs at once.
- Registration takes roughly one to three minutes per scan; COLMAP dense stereo about 28 minutes per stage on a 4 GB GPU.
- `--stage NAME` restricts P4 and P7 to one stage; `RUN_TAG=name` makes P4, the visibility check and P7 read `data/pointclouds_name` and write `data/matches_name` and `data/status_name`, so experiments never overwrite the main run.

- Keep before/after result tables in `results/experiments/` when changing the pipeline, and add a dated line to `docs/PROGRESS.md`.

9.3 Conventions

- Fixed random seeds everywhere; runs are exactly reproducible.
- Each phase writes its results to disk and prints the numbers; nothing is left only in memory.
- Plain NumPy, pandas, JSON and CSV; no configuration layers. Each script has a docstring explaining its method and why.
- Ground truth is for evaluation only.
- The git remote is on GitHub; pushing needs credentials that are not set up on the working machine.

10. Likely questions, with honest answers

Why simulate instead of using a real scan?

No public data set gives the design, a real scan, how they line up and which elements were built. With simulation the truth is exact and we can turn each real-world problem on separately and measure its cost. The price is that results are only as realistic as the simulation, which is why a real-data check is the top item in the future work.

Why match to the design instead of detecting walls in the cloud?

The design already says where every element should be, so the question becomes "are there points here?". The earlier blind approach (plane and cluster finding) matched very few elements in early stages.

Why does plain ICP fail?

It only looks for nearest points, so a scan that starts metres away or turned pairs with the wrong parts and settles in a wrong fit. Our toy example (Figure 4) shows it. In our benchmark it was right in 0 of 100 tests.

How do you know the pipeline is not peeking at the answer?

Ground truth is stored in separate fields and files and read only by scoring and diagnostic scripts. Registration, matching and visibility use only points, colours, camera positions and the design. The visibility check even uses the estimated registration, not the true one.

What does an average error of 2 points mean?

On average over the 16 stages, the estimated progress differs from the true progress by 2 percentage points; for example 58% instead of 60%. The worst stage was 5.7 points. With the learned correction the average is 0.7.

Are the easy-world error (0.8) and the hard-world error (1.95) comparable?

They use the same method and the same stages, but the hard world has a harsher sensor, a busy site and as-built errors, so the increase measures the cost of realism.

What is the 10% rule and is it cheating?

A detection with under 10% of the points expected for that element is rejected, because a few stray points from a neighbouring surface should not make an element look built. The expected count comes from the camera positions and the design, not from the answer key. The threshold was chosen on these runs, which is a limitation.

Why is 'uncertain' better than 'not started'?

If no camera could have seen an element, the lack of points says nothing about whether it is built. Counting it as not built would bias progress downward, and counting it as built would be a guess.

Why did colour not help?

Under changing light, most materials look the same grey, and clutter such as brick piles has the same colour as walls. Geometry was already enough to align every test case.

What did COLMAP teach us?

That photogrammetry clouds can be fed into the pipeline unchanged, that they need textured surfaces, and that on the finished building they lose windows and thin parts so much that the progress estimate falls 35 points short, while a perfect cloud from the same photos would score 94%.

What would you do first with more time?

Run on a real survey, and freeze a test split so the thresholds are not tuned on the data they are tested on.

What is the biggest risk in the results?

Over-optimism from simulation on one building with thresholds tuned on the same runs. We state this in Chapter 7.

Appendix A. Glossary

Term	Meaning
Answer key / ground truth	The exact list of what is built at each stage, from the manifests. Used only for grading.
As-built / as-planned	What is really built, against what the schedule or design says.
BIM	Building Information Modelling: a digital building model whose parts are objects with geometry and data.
Bundle adjustment	Jointly adjusting all camera positions and 3D points to minimise reprojection error.
Clutter	Everything in a scan that is not in the design: scaffolding, cranes, workers, material piles, trees, ground.
COLMAP	Open-source photogrammetry software (SfM and dense stereo).
Construction front	The highest storey that has many completed elements: where building is currently happening.
Count-weighted / volume-weighted	Progress where every element counts equally, or where elements count by their volume.
DART	The labelling tool, built on SAM3, used to find clutter in camera images.
Dense stereo (MVS)	Computing a depth for nearly every pixel by comparing photos; needs a GPU.
Detected	An element that received at least 8 scan points.
Element	One wall, slab, beam or column of the building.
FFT	Fast Fourier Transform: a fast way to compute a correlation for all shifts at once.
FPFH	Fast Point Feature Histograms: a 3D point descriptor used for feature-based registration; tried and replaced.
GlobalId	The 22-character unique ID of an IFC object.
GNSS / GPS	Satellite positioning; a drone survey is georeferenced by it with metre-level accuracy.
Heading	The compass direction; the main unknown rotation in registration.
Height coverage	Height spanned by an element's points divided by its true height; tells completed from partly built.
ICP	Iterative Closest Point: pair each scan point with its nearest model point, move the scan to reduce the gaps, repeat.
IFC	Industry Foundation Classes: the open file format for BIM.
KD-tree	A tree that splits points alternately along each axis so that nearest-point queries are fast.
LiDAR	Laser distance measurement; laser scanners produce dense, accurate clouds.
Manifest	The JSON file per stage listing which elements are built or partly built.
Normal	The direction a surface faces; used to tell walls (vertical) from floors (horizontal).
Occlusion	A thing in front hiding a thing behind it.
Point cloud	A set of 3D points on the visible surfaces of a scene.
Point-to-plane	An ICP distance measure to the local surface plane instead of to a single point.
Progress error	Estimated minus true progress in percentage points.
Ray casting	Shooting rays into a 3D model and recording the first hit; how we simulate scans.
Registration	Finding the transform that lays a scan on a model.
Reprojection error	The pixel distance between a projected 3D point and its observed feature.
Rigid transform	Rotation plus translation; nothing is stretched.

Term	Meaning
RUN_TAG	An environment variable that redirects P4, the visibility check and P7 to separate data folders for an experiment.
SAM / SAM3	Segment Anything models that cut out objects given prompts.
Segmentation	Labelling each pixel or point with what it belongs to.
SfM	Structure from Motion: solving camera positions and sparse 3D points from photos.
SIFT	A classic image feature detector and descriptor.
SLAM	Simultaneous localisation and mapping: building a map while tracking one's own position.
Stage	One of the 16 simulated construction states.
Status	Per element: completed, partial, not started or uncertain.
Survey frame	The coordinate frame a survey reports its cloud in (GPS-based or a scanner's start frame).
Tukey loss	A robust loss that ignores pairs whose distance is too large; makes ICP resistant to clutter.
Uncertain	Not detected, and the cameras could not have seen it: excluded from the progress number.
Visibility check	Asking whether the cameras would have seen an undetected element if it were built.

Appendix B. Per-stage results on the hardest test

Stage	True %	Est. %	Err.	Corr. %	90% range	In	Missed	False	Reg.
00 -1 fundering structure	6.7	6.7	-0.0	6.9	5.5 to 8.4	yes	3	6	1.7 cm
01 -1 fundering envelope	9.2	9.8	+0.6	9.5	8.0 to 11.0	yes	1	6	2.7 cm
02 begane grond structure	12.0	9.9	-2.1	11.5	10.0 to 13.0	yes	28	10	4.3 cm
03 begane grond envelope	23.4	24.7	+1.3	23.0	21.4 to 24.5	yes	18	6	2.5 cm
04 begane grond interior	29.9	30.3	+0.4	28.0	26.4 to 29.5	no	22	4	2.4 cm
05 eerste verdieping structure	36.8	35.7	-1.1	38.1	36.5 to 39.6	yes	20	1	4.0 cm
06 eerste verdieping envelope	49.4	50.1	+0.7	49.7	48.2 to 51.3	yes	14	5	1.7 cm
07 eerste verdieping interior	55.5	55.8	+0.3	54.1	52.5 to 55.7	yes	17	1	4.0 cm
08 tweede verdieping structure	62.1	59.2	-3.0	63.2	61.6 to 64.8	yes	31	6	2.0 cm
09 tweede verdieping envelope	71.0	69.7	-1.2	70.8	69.2 to 72.4	yes	20	5	1.4 cm
10 tweede verdieping interior	76.0	75.0	-1.0	74.8	73.2 to 76.4	yes	12	5	1.8 cm
11 derde verdieping structure	85.5	82.8	-2.8	86.3	84.7 to 87.9	yes	14	4	1.5 cm
12 derde verdieping envelope	87.8	84.6	-3.2	88.0	86.4 to 89.6	yes	26	3	1.2 cm
13 derde verdieping interior	93.9	88.1	-5.7	93.4	91.8 to 95.0	yes	50	3	1.9 cm
14 dak structure	96.8	93.7	-3.1	97.2	95.6 to 98.8	yes	25	3	1.4 cm
15 dak envelope	99.1	94.4	-4.7	98.5	97.0 to 100.1	yes	26	0	1.4 cm

Table 14. *Per-stage results (results/experiments/main_table_13_busy_late_misplaced_rule_on.csv): busy site, realistic sensor, GPS-sized offsets, late and misplaced elements, 10% rule on. Err. = estimate minus truth (points); In = truth inside the 90% range; Missed / False = built elements called not started / unbuilt elements called built; Reg. = registration error.*

Registration was flagged by its own checks at one stage (the second-floor interior), although it was within 1.8 cm of the truth: a false alarm, not a silent failure.

Appendix C. Script reference

Script	Purpose
p0_sanity_check.py	Element counts; GlobalId overlap check between the design and the as-planned files
p1_parse.py	IFC to bim_complete.csv
materials.py	IFC materials to 16 colour categories; catalogue colours the matcher knows
p2_ablate.py	The 16 stages and manifests (the answer key)
p2_p3_simulate_stages.py	Shared simulator helpers: ray casting, viewpoints, noise, jitter, survey offsets
p3_render.py	The scan simulator: sensor models, busy site, colour, DART views
site_clutter.py	Scaffolding, crane, fence, props, piles, workers, machines; off-design, late and misplaced elements
registration.py	Gravity levelling, floor-plan FFT search, height sweep, 3D verification, robust ICP, scale, flags
color_registration.py	Colour-aware wrapper around registration.py (clutter filter, near-tie ranking, coloured ICP)
p4_match_bim.py	Surface sampling, KD-tree matching with a margin, detection, height coverage
p4_visibility.py	Expected points per element and the 10% rule inputs
p7_score.py	Status, progress, error tables, error bars, figure
r5_calibrate.py	Fits the error-bar correction on one run
r3_registration_bench.py	Registration test bench (resumable per trial)
sfm_experiment.py	Photos, COLMAP, survey-frame fit, quality metrics, ideal cloud
dart_label.py, make_dart_figs.py, make_dart_pose_fig.py	DART clutter removal and its figures
p4_segment.py, p4_eval_segment.py	Earlier blind segmentation baseline and its scoring
make_media.py	All pictures, animations and 3D files in media/
make_presentation.py, make_endterm_presentation.py	The mid-review and end-term slide decks
make_report.py, make_report_figs.py	This report and its concept figures
export_*.py, *_viewer*.html, visualize_pointcloud.py	Browser viewers for the point clouds and segmentation

Appendix D. Project history from the git log

Commit	Author (as recorded)	Date	Summary
cc9adf1	Prem Suggu	2026-11-01	COLMAP experiment, old-vs-new comparison, improvement ladder
ed8132f	Sagar	2026-10-31	Interior site objects and DART clutter removal (one-stage test)
a622398	Prem Suggu	2026-10-30	Commit data/raw, data/matches and logs (no longer gitignored)
2b622cb	Prem Suggu	2026-10-30	Commit the design IFC so a fresh clone can run the pipeline
3115c89	Prem Suggu	2026-10-30	Colour in the full pipeline, R5 error bars, busy-site late/misplaced runs
c1c6227	Prem Suggu	2026-10-29	Add new CSV results for experiments and implement color registration
6c6e32b	Prem Suggu	2026-10-29	feat: add busy site simulation and clutter elements to construction site rendering
cd99269	Prem Suggu	2026-10-29	Add media/: pictures, animations and 3D files of the whole project
0102c94	Prem Suggu	2026-10-29	Realistic simulation + robust registration + visibility check (R2/R3)
d823a2c	sagar-sarvam	2026-09-18	Initial commit: BIM-based construction progress monitoring pipeline

Table 15. *Commits on the main branch. Who worked on which part can be read from the log; this report does not assign credit by name.*